How Community Feedback Affects Vibe Coding Creative Processes

ANONYMOUS AUTHOR(S)

Fig. 1. Overview of the CoVibe workflow and study design. Community feedback is incorporated into successive application revisions through bounded feedback selection and AI-mediated revision. A formative study informed the design of CoVibe, followed by a user study examining its use in practice.

Vibe Coding allows natural-language community feedback to enter AI-mediated application revision, creating a pathway from social feedback to executable change. We investigate how this pathway can be organized as collaborative work and how participants contribute to and interpret it across application versions. We first conducted formative interviews with 25 participants, identifying challenges around openness, implementation, limited revision opportunities, and contribution traceability. These findings informed CoVibe, a research platform for a bounded feedback-to-revision workflow. We then conducted an approximately eight-day deployment with 30 participants across two revision rounds. Analysis of 253 community contributions, 60 version transitions, platform activity,

2027. Manuscript submitted to ACM

provenance records, and post-study interviews shows that participants negotiated what was open to change, comments retained social meaning when becoming revision sources, selection bounded materialization, and executable versions made one realization of feedback inspectable while becoming the basis for further participation. We derive implications for collaborative Vibe Coding work.

CCS Concepts: • **Human-centered computing → Collaborative and social computing**.

Additional Key Words and Phrases: Vibe Coding; Community Feedback; Collaborative Creativity; Human-AI Interaction; Creative Processes

ACM Reference Format:
Anonymous Author(s). 2027. How Community Feedback Affects Vibe Coding Creative Processes. In *Proceedings of CHI Conference on Human Factors in Computing Systems (CHI '27).* ACM, New York, NY, USA, 35 pages.

1 Introduction

Andrej Karpathy introduced *Vibe Coding* in 2025 as building software through repeated natural-language interaction with a large language model while paying relatively little attention to generated code [33]. Subsequent studies characterize Vibe Coding as cycles of expressing intent, generating software, inspecting behavior, revising requests, and repairing failures [14, 18, 52, 60, 65]. In these cycles, desired behavior is specified in language, while executable results show how requests need revision [3, 38, 46, 76]. This extends end-user programming concerns with expressing computational intent beyond formal syntax [53, 59], while human–AI interaction research emphasizes continued interpretation and verification of generated outputs [1, 68]. For this paper, the important property is that a natural-language contribution can enter an AI-mediated application-revision pathway without first being rewritten as a code-level instruction.

Collaborative Vibe Coding distributes this language-to-execution cycle across people. Prior work examines shared prompts, cross-role prototyping, product-team workflows, and negotiations of intention and responsibility around generated software [17, 19, 44, 72, 75]. Participatory Vibe Coding also shows domain participants translating situated needs into specifications [35]. These settings largely organize collaboration among people already participating in development. Community feedback creates a different participation relation: a person can contribute through a social feedback channel without directly joining implementation. This distinction matters because comments can combine evaluation, diagnosis, suggestion, explanation, affect, and social positioning, and recipients interpret them in relation to their own goals and the developing work [20, 22, 40, 86]. In software practice, feedback also passes through interpretation, prioritization, and action before becoming a product change [47, 51].

Prior systems have already connected feedback to concrete revision: d.note links annotations with functional modifications to interface prototypes, while Synthia turns selected feedback into alternative drafts while preserving source relationships [26, 89]. The gap is therefore more specific than whether feedback can produce a revision. Vibe Coding allows selected community comments to enter the same natural-language revision channel used to modify an executable application, producing a version that re-enters community interaction. The same comment can retain evaluative or interpersonal meaning while also acquiring a role as a revision source. AI-mediated materialization adds another interpretive step: the generated version instantiates one possible realization of the comment and can expose consequences that its wording did not fully specify. Concrete externalizations have long been treated as resources for reflection and reframing in design [6, 66]. What remains less understood is how this feedback–revision–version–participation cycle can be organized as collaborative Vibe Coding and how participants contribute to and interpret it across application versions. We use this connection as a concrete setting for the broader questions:

RQ1: How can we understand and design for vibe coders' collaborative work?

RQ2: How do vibe coders work together creatively in a collaborative platform?

We first conducted formative interviews with 25 participants with prior generative-AI software-creation experience. Their accounts identified four challenges: sharing did not establish a common path to collaborative development; feedback remained separate from implementation; limited revision opportunities required choices among different directions; and external contributions were difficult to connect to later versions. These findings informed CoVibe, a research platform that operationalizes a bounded feedback-to-revision workflow. Figure 1 provides an overview of the CoVibe workflow and the two studies used to inform and examine this form of community-supported application revision. We conducted an approximately eight-day exploratory deployment with 30 participants in a bounded asynchronous community. Each participant created one runnable application and contributed to others. Two rounds of community interaction followed by AI-mediated revision produced 60 transitions across V0–V1–V2. We analyzed 253 contributions, selection and provenance records, executable versions, platform activity, and application-centered interviews.

Our findings trace a linked process. When comments could cross from social communication into executable revision, participants negotiated when applications were ready for external involvement and which aspects were open to change. Comments retained social, evaluative, and cautious meanings even when some also became revision sources. Because only a subset could enter each revision, selection allocated materialization opportunities without serving as a measure of comment value; unselected comments could remain influential. AI-mediated revision then made one realization of selected feedback inspectable in a runnable version. Unexpected consequences could become material for further thinking, while unwanted consequences could make authorial boundaries more explicit.

This work contributes an empirical and design account of collaborative Vibe Coding in which community feedback can move from social exchange into AI-mediated executable revision. It characterizes the design challenges involved in organizing this transition and explains how creators and contributors interpret consequential comments, bounded materialization opportunities, and concrete revised versions across iterative development.

2 Related Work

Our work brings together two bodies of research that address different parts of the process examined here. Research on Vibe Coding and natural-language programming explains how people express development intentions, inspect generated software, and coordinate revision with others. Research on creative and software feedback explains how external contributions are produced, interpreted, selected, and incorporated as work evolves. We use these literatures to position a collaborative configuration in which community comments can enter AI-mediated application revision and the resulting executable versions can become the basis for subsequent participation.

2.1 Participation in Collaborative Vibe Coding

Empirical studies of Vibe Coding characterize development as an iterative movement among intention, natural-language requests, generated outputs, and evaluation. Sarkar and Drosos describe a conversational process in which people set goals, generate code, test results, and refine subsequent requests [65]. Meske et al. examine how developer intent is mediated through dialogue with generative AI [52], while Chou et al. document practices ranging from broad reliance on generated outputs to closer inspection, debugging, and refinement [14]. Across these studies, executable results do not simply complete a request; they provide information that users interpret when deciding how development should continue.

This movement between request and result also appears in earlier studies of code-generating tools. Barke et al. describe programmers moving between using generated code for an intended task and exploring possible solutions [3].

Studies of conversational programming assistants examine how users express requests, inspect responses, and continue developing solutions through dialogue [63]. Vaithilingam et al. identify difficulties in understanding, editing, and debugging generated code [76]. Across these studies, generated outputs require users to judge whether a result works and how well it reflects the intended change. Together, this earlier work shows that generated outputs remain objects of judgment and repair, making interpretation part of the development process even when code generation itself is automated.

Collaboration introduces additional people into this cycle of proposing, interpreting, and evaluating changes. CoPrompt supports sharing, referring to, and reusing prompts during collaborative natural-language programming [19]. Subramonyam et al. examine how software teams prototype generative-AI features through prompt experiments, output evaluation, and interface design [72]. Li et al. show how Vibe Coding enters product-team practices around ideation, prototyping, collaboration, trust, and responsibility [44]. Together, these studies show that collaboration in natural-language development can involve shared intentions, prompts, evaluations, and decisions about generated results.

A broader view of participation appears in end-user software engineering [37], co-design [64], and participatory design [70], where people contribute through knowledge of needs, contexts, and use. Recent Vibe Coding studies bring similar forms of participation into work with executable applications. Kim et al. describe nurses moving from workplace problems to specifications tied to screens, triggers, and failure conditions [35]; Gama et al. examine novice programmers' use of multiple tools and their continued assessment of generated outputs [23]; and Kobiella et al. investigate prototyping across different levels of technical experience [38]. These studies broaden the range of people who can shape application development and the kinds of knowledge that can enter the process.

The circulation of runnable applications extends this collaborative picture beyond an immediate development group. Replit, v0, and Lovable support publishing runnable applications [48, 62, 77]; Thinklet combines application discovery, comments, and remixing [74]; and LingGuang supports creating and sharing interactive applications [2]. Earlier research on remix communities helps clarify how participation changes once artifacts are shared more widely. Studies of Scratch have examined learning through remixing [16], relationships between remix diffusion and originality [29], and creators' responses when others reuse their work [30]. Cambre and Klemmer similarly study how students build on peers' projects through remixing [7]. These studies distinguish several ways of taking part in another person's work, including viewing, discussing, reusing, and modifying it. As vibe-coded applications become more visible and shareable, community feedback becomes part of the broader social context around them, connecting research on collaborative application development with a longer line of work on feedback and revision.

2.2 Community Feedback in Creative Revision

Feedback research offers a complementary account of how outside contributions become part of an evolving work. Xu et al. follow successive rounds of crowd feedback and design revision [84], and Yen et al. show how feedback review can prompt people to recall goals, question existing choices, and prioritize changes [86]. Foong et al. describe how novice and expert designers draw on their own knowledge when interpreting critique [21]; their broader framework of online feedback exchange traces activities across help-seeking, sense-making, and action [20]. In software development, Li et al. document how practitioners collect, consider, and act on user feedback across a broader feedback lifecycle [47]. Across this work, the influence of feedback is shaped by how it is interpreted, weighed against other concerns, and connected to later revisions.

How a contribution is phrased and presented can shape that process before any change is made. Voyant and CrowdCrit organize observations and critiques from non-experts [50, 83]; Critiki scaffolds crowdworkers in producing design feedback [24]; and Critique Style Guide helps contributors revise comments through language-based guidance and examples [41]. Lekschas et al. show that question- and statement-based feedback can lead to different revision outcomes [43], while Crain and Bailey compare how feedback presentation formats affect the way recipients work with critique [15]. These studies connect the language and structure of feedback with how recipients interpret its meaning and relevance to subsequent revision.

In community settings, feedback is also shaped by the social relationship between contributors and recipients. Xu and Bailey study expectations in an online critique community [82]; Cheng et al. examine public requests for critique [10]; and Kou and Gray describe critique as collective interpretation and sensemaking [40]. Other studies show that participation changes with the stage of the work and characteristics of comments [25], that affinity and trust shape feedback exchange [9], and that the source of feedback affects how recipients experience and use it [58, 80]. Nguyen et al. further show that affective language and source can influence critique reception and later work [56]. This work shows that community feedback can carry information about the artifact together with cues about tone, relationship, and social intent, which can shape how a contribution is understood.

Several studies follow external and audience contributions beyond the moment they are given and examine how they enter later creative decisions. In livestreamed indie game development, viewers and prospective players contribute ideas and feedback during production [45]. Audience in the Loop documents how comments, reposts, and memes become part of later decisions in micro-drama production [8]. Crowd-based creative systems have explored related questions of how many contributions can be organized within a continuing process. Ensemble combines a leader's creative direction with crowd contributions [34]; Mechanical Novel structures writing through reflection, goal setting, and revision tasks [36]; and IdeaHound helps groups organize and explore relationships among ideas [69]. Young-Ng et al. examine how human agency and AI autonomy can be distributed during idea selection [87]. Across these examples, contributions can remain part of the creative process through the ways they are selected, organized, and carried forward.

Several systems connect expressed feedback more directly to concrete changes in a developing artifact. d.note combines annotations with functional modifications to running interface prototypes [26], while ProtoChat connects crowd testing and feedback on conversational components with subsequent design revisions [13]. Generative systems extend this link by turning people's descriptions into new representations or artifact states. UrbAI helps citizens visualize urban-development ideas [61], and FIERO uses generative AI to combine participants' contributions during collaborative storytelling [90]. In writing, Friction supports writers in breaking feedback into manageable units and planning revisions [88]. Synthia allows writers to select and combine feedback into alternative drafts while preserving links among comments, source text, and versions [89]. These systems make expressed ideas available in a concrete form that can itself become the subject of further inspection and response.

Once contributions are carried into later versions, the history of those changes also becomes part of collaborative work. Kaleidoscope brings project artifacts, documentation, and peer feedback into a shared space [71], while DocuViz visualizes how individual contributions develop across a document's revision history [79]. Research on remix communities has examined how automatic attribution and human-given credit are understood when people build on one another's work [54]. Edits & Credits similarly studies how integration and attribution operate in online creative collaboration [49]. This work shows how records of change can help participants connect earlier contributions with later artifact states and understand how individual input becomes part of a collective outcome.

A nearby body of work examines audience perspectives within AI-mediated creative workflows. Proxona builds interactive audience personas from audience data [12]; Writer-Defined AI Personas supports feedback from specified reader perspectives [4]; SimTube generates possible audience comments on videos before publication [32]; and PosterMate organizes discussion among audience persona agents and can apply agreed edits to a poster [67]. These systems center on creators' interactions with generated or modeled audience perspectives. Our setting centers on feedback from people engaging with the same shared applications, with creators and contributors participating in the same social process.

Research on collaborative Vibe Coding has mainly examined creators, teammates, and domain participants who are already involved in application development. Research on creative feedback has shown how external contributions are expressed, interpreted, selected, and incorporated into ongoing work. Less is known about community settings in which feedback can enter an AI-mediated revision process and contribute to the continued development of a shared executable application. This work examines collaborative Vibe Coding at this intersection, where community feedback can become part of the development of shared applications.

3 Formative Study

To understand how vibe coders involve others in application development, we conducted a formative interview study with experienced vibe coders. We examined how participants collaborated around evolving applications, how contributions from others entered their development process, and what difficulties arose in coordinating those contributions across iterations. These accounts informed the design goals for a research platform for investigating community participation in collaborative Vibe Coding. This study was approved by the IRBs of all organizations involved.

3.1 Method

Semi-structured interviews lasted 45–60 minutes and addressed participants' prior projects, exchanges with others, assessment of suggestions, and revision work. We presented no prototype, concept scenarios, or feature alternatives. With informed consent, interviews were recorded, transcribed verbatim, and anonymized; three researchers checked transcript accuracy. We conducted thematic analysis following Braun and Clarke [5]. Following recommendations for human–AI collaboration in thematic analysis [85], we used GPT-5.6 to organize de-identified excerpts, suggest preliminary codes, and compare patterns. Suggested codes were linked to source excerpts. Four researchers reviewed and refined candidate themes against the transcripts and synthesized four design challenges. The research team retained responsibility for all interpretations; model outputs served as analytical aids throughout.

3.2 Participants

We recruited 25 participants, identified as P1–P25: 14 through social media and 11 offline. Eligibility required participants to be at least 18 years old and to have used generative AI to create or substantially modify software. Social-media recruits had previously shared vibe coding projects or experiences online. Participation was voluntary and uncompensated.

Participants ranged in age from 21 to 40 years ($M = 27.28$); 14 identified as male and 11 as female. Sixteen participants were based in China, while the remaining nine were based in Canada, Japan, South Korea, the United States, or Hong Kong, China. Their backgrounds included computer programming, design, art, product development, education, game development, engineering, and related fields. Programming experience ranged from 0 to 20 years, including seven participants who reported no prior programming experience, while prior vibe coding experience ranged from a small number of projects to 15–20 projects.

Table 1. Formative study participant characteristics ($N = 25$).

PID	Age	Gender	Background	Programming experience (years)	Number of prior vibe coding projects	Location
P1	22	Male	Computer Programming	4	10	China
P2	24	Female	Computer Programming	7	≤ 5	China
P3	27	Female	Design, Computer Programming, Art	9	5–10	United States
P4	22	Male	Computer Programming, Engineering	3.5	≤ 5	South Korea
P5	40	Male	Computer Programming, Product Development	20	10–15	China
P6	22	Male	Other	5	≤ 5	United States
P7	28	Female	Computer Programming	5	15–20	China
P8	28	Male	Design, Game Development, Art	0	≤ 5	China
P9	25	Male	Game Development, Web Development	9	5–10	HK, China
P10	21	Female	Design, Game Development, Art	0	≤ 5	China
P11	22	Female	Design, Art, Game Development	0	5–10	China
P12	25	Female	Design, Web Development, Engineering	3.5	≤ 5	China
P13	29	Male	Computer Programming, Product Development	4	10–15	HK, China
P14	25	Male	Design, Computer Programming	0	≤ 5	China
P15	23	Male	Computer Programming, Education, Writing	1	≤ 5	China
P16	32	Male	Other	0	≤ 5	China
P17	30	Male	Computer Programming	5	≤ 5	China
P18	32	Female	Software Engineering, Spatial Data Analysis	8	5–10	China
P19	28	Female	Computer Programming, Engineering	10	15–20	Canada
P20	22	Female	Engineering, Computer Programming	4	5–10	Canada
P21	32	Male	Computer Programming	8	5–10	Japan
P22	33	Male	Product Development, Computer Programming	5	5–10	China
P23	30	Female	Product Development, Art, Design	0	15–20	China
P24	30	Male	Game Development	14	≤ 5	United States
P25	30	Female	Art, Design, Education, Product Development	0	≤ 5	China

Note. Background categories follow participants' questionnaire responses. Prior project counts are shown as reported, including ranges.

Participants had created a range of applications, including websites, games, interactive prototypes, visual experiments, research tools, and engineering software. They reported using tools such as Codex, Claude Code, Cursor, VS Code, Google AI Studio, and other conversational or agentic programming tools. Table 1 summarizes participants' demographics, backgrounds, programming experience, prior vibe coding project counts, and locations.

3.3 Design Challenges

3.3.1 Sharing Did Not Define the Extent of External Involvement. Participants shared applications for exposure, feedback, and further development, with different expectations of others' involvement. P6 had open-sourced their code and welcomed modifications, yet used Douyin and Xiaohongshu to attract feedback and WeChat for deeper discussion. They valued private conversations for candid responses and suggestions about different directions. P15's plugin developed through personal use and friends' testing; Xiaohongshu supported sharing and tutorials, while GitHub handled distribution and issues. Creators connected these channels to their development work.

More direct involvement required additional arrangements. A player offered to improve the interface of P24's non-commercial RPG. P24 agreed and shared the code; the player subsequently extended its characters and story. Private discussions coordinated these contributions. P17, meanwhile, shared a game to gauge market interest and

Fig. 2. Current feedback flow in vibe-coded application development and the four breakdowns identified in our formative study. Feedback typically moves from application sharing and community response to creator-led interpretation, selection, and implementation, while limited permission, separation between commenting and implementation, constrained revision opportunities, and weak contribution traceability hinder its direct participation in application development.

recruit collaborators for art and interface work while retaining responsibility for core gameplay. Although the group reached around 20 members, P17 reported that none had joined the project. Some sought help with their own projects without contributing to P17's.

These accounts distinguish making an application available, seeking responses, and involving others in its development. P6's openness to code modification coexisted with separate channels for discussion, while P17 sought contributions to particular aspects of the game. External involvement could therefore differ in both form and scope. Sharing alone left the scope and organization of participation to subsequent exchanges and arrangements.

3.3.2 Feedback Remained Separate from Implementation. Creators' accounts distinguished recognizing useful input from implementing it. P2 incorporated feedback into a doctoral-application tool's matching logic, school recommendations, and research-group links. Interface criticism remained difficult to address: *"It's not that [the feedback] was unimportant; I just couldn't handle it, because I don't know frontend development."* Recognizing the problem did not supply the capabilities needed to resolve it.

P5 described a different separation in previous professional work on Duokan, a Chinese e-book reading platform. Requests judged worth implementing entered a requirements repository and received a development timeline: *"If it is a valid requirement, it goes into my requirements repository, and I assign it a timeline."* Acknowledging a request therefore preceded the work of implementing it.

P20 described a related implementation difficulty while using AI to convert a colleague's code into a smaller program. Some suggested modifications produced *"no optimization at all"* when run, requiring inspection of the implementation. Generating a modification still left work to establish its actual effect.

Feedback could identify a recognized problem or enter a development plan while remaining separate from a working revision. These separations involved different kinds of work: P2 faced an implementation difficulty, P5 described the scheduling of accepted requests, and P20 needed to inspect generated changes. Implementation and assessment constituted additional work between receiving input and determining what it would mean for the application.

3.3.3 Incorporating Suggestions Required Choices About Development Directions. External suggestions varied in their relevance to an application's direction. Drawing on product-management experience, P13 described requests as *"scattered, specific, and sometimes contradictory."* They warned that implementing every suggestion could produce an increasingly bloated product and weaken its focus. A reported problem also did not necessarily identify the most appropriate solution. Assessing feedback involved relating individual requests to broader development priorities.

P23 described a team disagreement about an AI workflow. Their approach generated the final result directly and was simpler for newcomers; colleagues proposed intermediate steps offering greater control. *"We would discuss whether the project needed speed or controllability, and then vote."* Both proposals addressed the task, but emphasized different priorities. The decision depended on which priority the team considered more appropriate for that project.

P25 used Claude Code to generate and test versions when others proposed alternative directions. They explained that *"the convenience of the technology has brought the cost of verification down,"* making them more willing to consider suggestions. Testing also supplied grounds for later decisions: when a similar proposal recurred, they could explain that they had already tried it and found it unsuitable. Here, the creator used AI to examine another person's idea and retained responsibility for deciding what the test implied for development.

Across these accounts, incorporating suggestions involved assessing their fit, choosing directions, and developing grounds for those choices through discussion or testing. Multiple ideas required creator judgment even when AI made alternatives easier to try.

3.3.4 Tracing Development Required Connections Across Input, Changes, and Versions. Participants' accounts raised related questions about continuity across projects, visibility of modifications, and the connection between comments and development activities. P7 described lightweight projects that ended after sharing. Subsequent ideas could lead to reusing code or starting another project, so continued development did not always take the form of successive versions of the original application.

Within continuing projects, P23 found frequent AI-assisted modifications difficult to track. Fixing one issue could affect previous changes, and they wanted clearer version comparisons to understand what had changed. Their account concerned the visibility of implementation changes across iterations.

P1 emphasized the connection between external input and development, describing the importance of understanding *"how a comment becomes an issue, and then how it becomes a branch."* This account connected a social contribution to subsequent development activities, identifying relationships beyond the sequence of application states.

Together, these accounts informed a traceability challenge: following application development involves understanding both the changes between versions and the input associated with those changes. A version sequence provides a record of states; connecting suggestions to development activities provides a basis for interpreting external involvement.

3.4 Design Goals

Our formative study identified four design challenges in how feedback participates in vibe-coded application development. These challenges appeared across the feedback-to-revision process, from sharing and commenting to implementation and attribution, as summarized in Figure 2. We translate these challenges into the following design goals. The design goals and their implementation in CoVibe are summarized in Table 2. The specific mechanisms and study configuration are described in Section 4.

DG1: Enable shared application development. Sharing, feedback exchange, and direct development involved different arrangements (Section 3.3.1). We aim to establish a shared process through which people can experience published

applications and contribute to their subsequent versions. This process should make the roles of creators and contributors clear and support continued participation beyond the initial act of sharing.

DG2: Bridge community feedback and application revision. Recognizing useful suggestions and implementing them required distinct work (Section 3.3.2). We aim to enable selected community comments to serve as instructions for AI revision of shared applications. This connection allows us to examine how participants move between discussing an idea and encountering its implementation. Section 4.2 describes how CoVibe connects selected comments to application revision.

DG3: Support participation under limited revision capacity. Participants assessed competing suggestions through development priorities, discussion, and testing (Section 3.3.3). These accounts motivated organizing participation when only some proposed directions can receive implementation opportunities. We aim to accommodate community support while giving creators limited influence over the direction of their applications. The two-round limit is a study configuration used to examine this goal; the interviews informed the need to choose among directions rather than a fixed number of revisions.

DG4: Make application evolution traceable. Participants highlighted the need to follow application changes and connect them with external input (Section 3.3.4). We aim to preserve relationships among comments, selected revision inputs, and published versions. These relationships should let participants identify the suggestions associated with a version and inspect the resulting application, without assuming that every selected suggestion was implemented.

4 Design

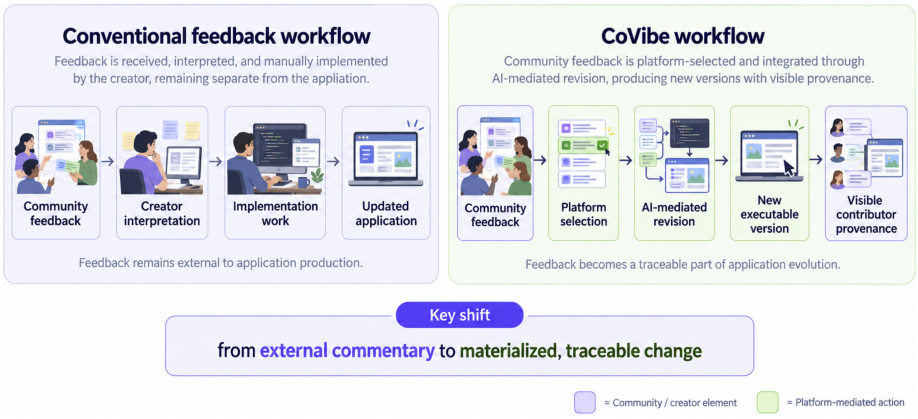

Fig. 3. Left: creators interpret community feedback and direct further development, potentially using AI tools. Right: CoVibe uses selected comments as inputs for AI-mediated application revision and retains their relationships to generated versions.

Building on the formative findings and design goals in Table 2, we developed CoVibe around a central mechanism: selected community comments serve as instructions for AI to generate revised application drafts. CoVibe provides a research setting for examining how participants understand and adapt to this connection between commenting and implementation.

Table 2. Design challenges, design goals, and design implementation in CoVibe.

Design Challenge	Design Goal	Design Implementation
Sharing did not establish a common path to collaborative development	DG1: Enable shared application development.	Supports application publication, gallery exploration, and asynchronous contributions to successive versions (Section 4.1).
Feedback often remains separate from implementation	DG2: Bridge community feedback and application revision.	Supports optional comment synthesis and uses selected comments as inputs for AI-mediated application revision (Section 4.2).
Limited revision opportunities require choices among different directions	DG3: Support participation under limited revision capacity.	Combines a like-weighted lottery with an optional creator wildcard to select a development direction for each application in each round (Section 4.3).
Feedback is difficult to connect with application versions	DG4: Make application evolution traceable.	Displays comment source links, selection indicators, and version lineage connecting selected inputs to published versions (Section 4.4).

Prior work has explored feedback-driven revision in which writers select feedback and use LLMs to generate revised drafts [89]. We investigate this process in a community setting. Figure 3 contrasts CoVibe with creator-led revision, in which creators interpret community feedback and translate it into development instructions, potentially using AI tools. In CoVibe, selected comments enter the AI revision process directly. We refer to this process as feedback-driven, AI-mediated application revision.

4.1 Supporting Shared Application Development

Participants act as creators of their own applications and contributors to others' applications (DG1; Figure 4). To create an initial version, creators describe requirements in natural language, inspect the AI-generated result, and refine it through dialogue. Published applications appear in a masonry gallery with previews, descriptions, and creator identities.

Contributors can try applications directly and comment on their experience (Figure 5). Comments, replies, likes, and saved comments support participation at different times. Participants can thus propose changes to others' applications while receiving feedback on their own.

In the study, creators chose their application topics. Publication of the initial version was followed by two community revision rounds: V0 served as the base for V1, and V1 for V2. Each release provided an updated application to try and a basis for further suggestions.

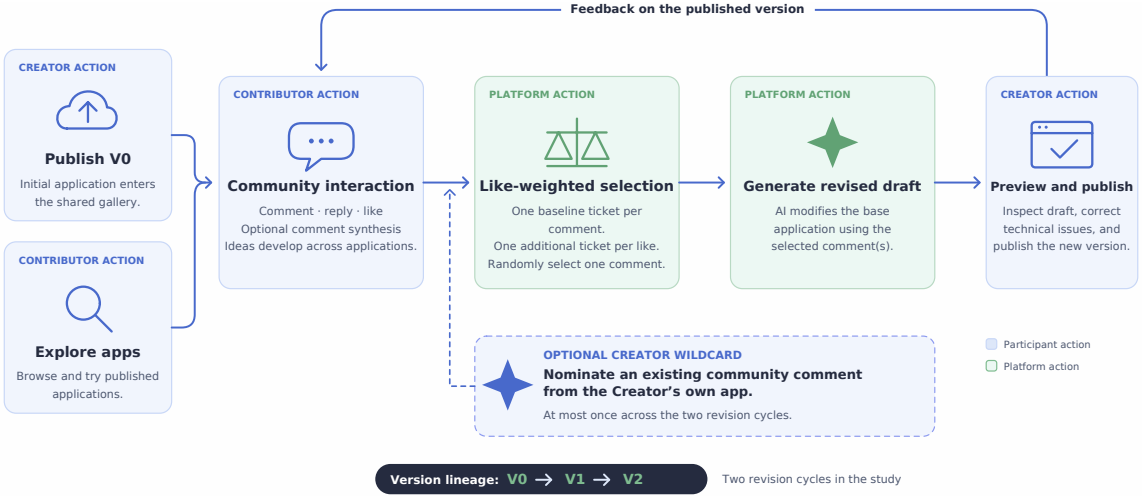

Fig. 4. Community revision workflow. Publication is followed by community interaction, comment selection, AI generation, and preview and publication of the next version. Interaction includes trying applications, commenting, replying, liking, and optional comment synthesis. A creator may nominate a wildcard comment before the lottery. Each published version becomes the next round's base, following V0 → V1 → V2.

4.2 Connecting Community Feedback to Application Revision

Community comments allow participants to propose application changes (DG2). Foong et al. [21] argue that online feedback systems should support designers in obtaining feedback, interpreting suggestions, and incorporating them into revisions. This perspective informs CoVibe's connection between discussion and implementation: participants develop suggestions through discussion and optional comment synthesis, and the platform passes selected comments to AI to generate revised drafts.

After trying an application, contributors can describe their experience, identify problems, or suggest changes. In their study of a UX design community on Reddit, Kou and Gray [40] found that creators and critics used follow-up questions and explanations to supply context, clarify design problems, and develop suggestions. Accordingly, CoVibe supports replies through which participants can clarify a suggestion's meaning, describe its use context, or explain an intended change.

Participants can also write synthesized comments. They select one or more comments on the idea-evolution canvas and rewrite, elaborate, or combine their content in an editable proposal that retains references to its sources. When composing a synthesis, participants can select comments they previously saved, including comments from other applications. Source links preserve the original authors and application contexts (Figure 5(d)).

This opportunity to integrate different perspectives draws on Harvey's [27] account of creative synthesis. A synthesized comment expresses its author's interpretation of the source ideas. Synthesis is optional; both ordinary and synthesized comments can be selected for development.

The mechanisms in Section 4.3 determine which comments enter AI revision. The platform supplies these comments with the base application to generate a draft. Creators can preview the generated draft and make technical corrections

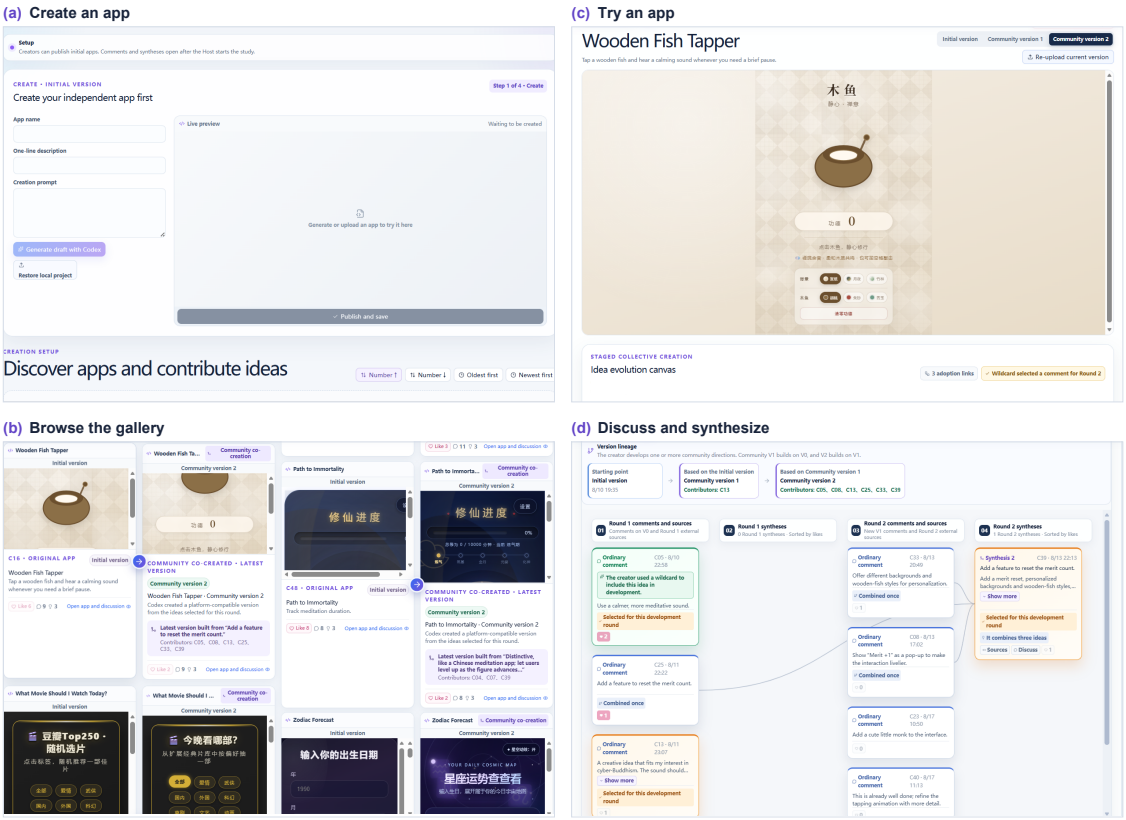

Fig. 5. CoVibe interfaces. (a) Creation workspace before generation, with natural-language requirements and a preview area. (b) Gallery pairing initial applications with their latest community versions. (c) Application preview and version switching. (d) Discussion and synthesis canvas showing comments, source links, selection indicators, and version lineage across two revision rounds.

when necessary before publishing it as a community version, but they cannot add new creative requirements or redirect the revision at this stage.

4.3 Supporting Participation under Limited Revision Capacity

CoVibe combines a like-weighted lottery with an optional creator wildcard to select each application's development direction for the round (DG3; Figure 4). At the end of each round, the study host locks comments and likes and initiates selection using the recorded counts.

Each comment receives one base ticket and one additional ticket per like. The system draws one ticket uniformly from the pool and selects its associated comment. If all comments have zero likes, they have equal selection probabilities.

In a randomized experiment, Muchnik et al. [55] found that an initial up-vote on a comment increased subsequent users' likelihood of up-voting it, allowing an early rating advantage to accumulate. This herding effect motivated our use of a weighted lottery rather than selecting the most-liked comment. Likes increase a comment's selection probability, while the base ticket preserves a chance for every comment. The lottery thus incorporates community support while allowing less-liked suggestions to enter development.

Each creator has one wildcard opportunity across the two revision rounds. They can nominate a community comment on their own application as an additional revision input. Before the draw, the platform removes that comment from the pool to prevent duplicate selection, then supplies it to AI alongside the lottery-selected comment. Creators thereby influence development through suggestions already submitted to the community. In Figure 5(d), green cards indicate creator-nominated wildcard comments, and orange cards indicate comments selected by the platform's lottery.

4.4 Making Application Evolution Traceable

The idea-evolution canvas displays relationships among comments, revision inputs, and application versions (DG4). Ordinary and synthesized comments occupy separate columns for each round. The second round's columns open when the study enters that round. Links connect syntheses to their source comments, and replies remain attached to their parent comments (Figure 5(d)). Participants can follow these relationships to revisit the suggestions behind a synthesis.

Each published community version records its base version and selected revision inputs. The gallery pairs initial and latest versions, while the application page provides access to earlier versions and development records.

This design draws on history flow, which visualizes document revisions through retained text and authorship [78]. CoVibe records selection and source relationships at the level of comments and application versions. Participants can identify a version's inputs, try the application, and compare the selected suggestions with the resulting changes. Contributors receive a notification naming the application and round when their comments enter development.

4.5 Technical Architecture

CoVibe uses React and TypeScript for its client, Node.js/Express for backend APIs, and SQLite to store applications, comments, source relationships, development tasks, and versions. The platform invokes Codex with GPT-5.6 Sol on the server side, providing the base application, application description, revision requirements, and selected community comments as inputs to generate a revised draft. Participant-written syntheses supply revision requirements; their source links remain in the database.

Before saving a draft for preview, the server checks that the output is nonempty, falls within size limits, and contains basic opening and closing HTML tags. Creators can inspect application behavior in the preview and correct technical failures when necessary, but cannot introduce additional creative requirements during this review.

5 User Study

Using CoVibe, we conducted an approximately eight-day deployment study with 30 participants to examine how they engaged with community feedback as input for AI-mediated application revision. We investigated contributions, application changes, and participants' interpretations through platform records, application version histories, interviews, and post-study ratings.

5.1 Participants

We recruited 30 participants through social media, university groups, and in-person recruitment among laboratory colleagues. Participants were eligible if they had previously completed at least one vibe-coded project, ensuring prior hands-on experience with Vibe Coding. Each participant received a US$10 gift card.

Participants represented backgrounds including design, art, HCI, computing and software development, statistics, marketing, communication, literature, and biology. Among the 24 who reported age, ages ranged from 20 to 38 years ($M = 24.67$, $Md = 24$). Eight participants had created one vibe-coded application, 11 had created 2–5, five had created

Table 3. Participant characteristics in the deployment study ($N = 30$).

Participant	Age	Background	Vibe coding duration (months)	Number of prior vibe coding projects
C01	28	Fine Art	3–6	> 10
C02	23	AI Design Research	> 12	> 10
C03	24	Art and Technology	1–3	≤ 5
C04	24	Design	> 12	≤ 5
C05	–	Art, Design	< 1	≤ 5
C06	24	Design	> 12	> 10
C07	–	Design	1–3	≤ 5
C08	25	Pet care	1–3	≤ 5
C09	–	Marketing	> 12	6–10
C10	21	Design and Engineering	> 12	> 10
C11	–	AI Coding, Data Agents	3–6	≤ 5
C12	23	Digital Media	> 12	6–10
C13	28	Applied Statistics	< 1	≤ 5
C14	–	Computer Science	6–12	≤ 5
C15	22	Design	< 1	≤ 5
C16	21	Animation Production	3–6	≤ 5
C17	25	HCI	3–6	≤ 5
C18	–	AI Development	6–12	6–10
C19	38	Fire Protection	6–12	6–10
C20	25	Software Engineering	> 12	> 10
C21	20	Chinese Language	3–6	≤ 5
C22	32	Interaction Design	6–12	≤ 5
C23	27	Design Studies	1–3	≤ 5
C24	26	HCI	> 12	> 10
C25	25	Internet Industry	< 1	≤ 5
C26	21	Computer Science	1–3	6–10
C27	20	Computer Science	1–3	≤ 5
C28	24	Media Communication	1–3	≤ 5
C29	26	Biology	3–6	≤ 5
C30	20	Design	1–3	≤ 5

Note. Participants are assigned pseudonyms C01–C30 throughout the manuscript. "–" indicates age not reported.

6–10, and six had created more than 10. Reported vibe coding experience ranged from less than one month to more than one year. Table 3 summarizes participant characteristics.

5.2 Procedure

The study was conducted remotely and asynchronously over approximately eight days. Figure 6 summarizes the end-to-end study workflow, which consisted of an initial application creation stage followed by two rounds of community interaction and AI-mediated revision. This study was approved by the IRBs of all organizations involved. Before the deployment, participants completed a background questionnaire and attended a 20-minute online introduction to the study tasks and platform workflow.

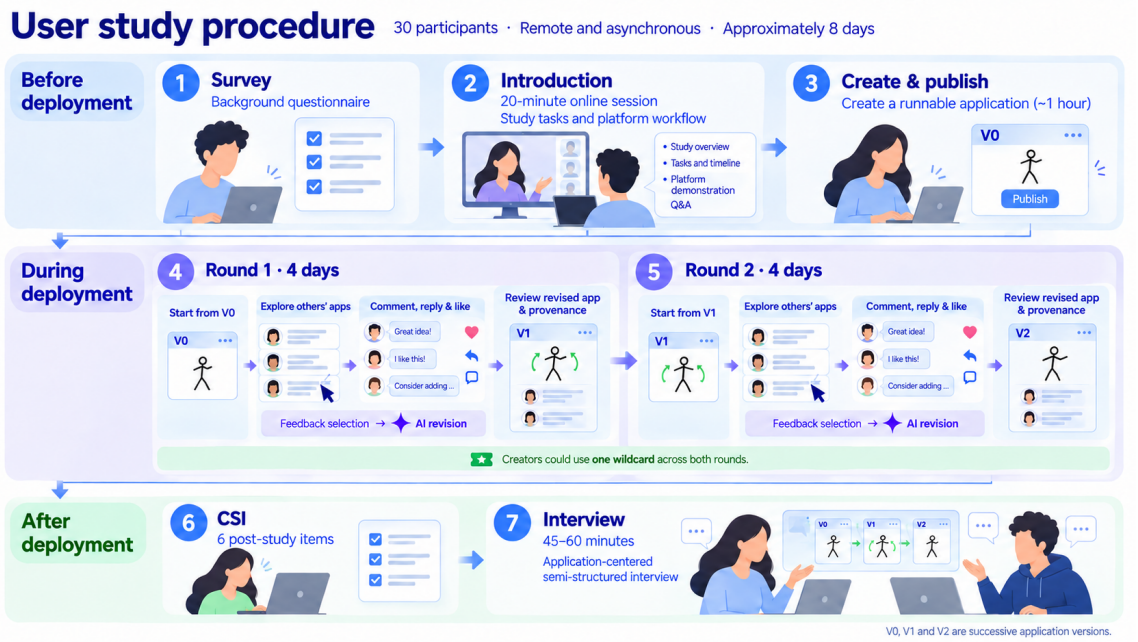

Fig. 6. Overview of the user study procedure. Before deployment, participants completed a background survey, attended an introduction session, and created and published an initial application (V0). During deployment, they participated in two four-day rounds of community interaction, including exploring applications, commenting, replying, liking, and reviewing subsequent AI-mediated revisions (V1 and V2) and their provenance. Creators could use one Wildcard across the two rounds to carry an additional community contribution into revision. After deployment, participants completed six adapted CSI items and a 45–60 minute semi-structured interview.

Participants first created and published one runnable vibe-coded application as V0 through multi-turn natural-language interaction with the system. They were instructed to complete the initial creation within approximately one hour, with no restriction on topic or form.

During Round 1, participants browsed and experienced others' V0 applications and contributed through comments, replies, likes, and synthesized comments. Participants were instructed to engage deeply with at least three applications created by others across the deployment, while they could browse and contribute to additional applications as they wished. At the end of the first four-day period, the research team closed Round 1 and initiated the first selection and revision cycle. For each application, the platform selected one community comment using the like-weighted stochastic mechanism described in Section 4. Across the two rounds, each Creator could use the wildcard once, in either round, to add one community comment on their own application to the revision source set.

Creators inspected the resulting version and provenance before release. They could correct technical failures but could not introduce additional creative requirements during this review. The released version became V1, after which Round 2 followed the same interaction structure. The same selection and revision procedure was then applied to V1 to produce V2. Thus, V0, V1, and V2 denote successive application versions rather than experimental conditions.

Researchers provided support for platform access and technical problems when necessary but did not guide feedback contributions, wildcard decisions, or creative directions. After the deployment, participants completed six post-study CSI items and participated in a 45–60 minute application-centered semi-structured interview.

5.3 Data Collection

We collected application-centered interviews with all 30 participants, behavioral logs, 253 textual community contributions (227 ordinary comments, seven replies, and 19 synthesized comments), platform selection and Wildcard records, AI-mediated revision records, applications and version histories, and responses to six post-study CSI items. Across 30 applications, we retained 90 runnable versions (V0, V1, and V2), corresponding to 60 consecutive revision transitions.

The platform maintained provenance linking contributions, selection events, wildcard use, AI-mediated revision records, and resulting application versions. Interviews revisited participants' actual applications, contributions, selection outcomes, wildcard decisions, and version histories to ground discussion in specific study events. The complete interview protocol is provided in the supplementary materials.

5.4 Data Analysis

We analyzed the data across four components: community contributions and initial applications, application revisions and trajectories, platform activity and post-study ratings, and participants' interpretations.

Community Contributions and Initial Applications. We analyzed all 253 textual community contributions using directed qualitative content analysis[31], with each complete contribution as the unit of analysis. Drawing on prior work on distributed online critique and revision-oriented feedback [40, 81], we developed a multi-label scheme with two dimensions: *communicative functions*, describing what a comment did within the interaction, and *revision-oriented features*, describing the forms of revision input it contained. Communicative functions included evaluation, direct recommendation, social or phatic response, comparison or reference, and process diagnosis. Revision-oriented features included issue identification, suggested directions, concrete solutions, specific referents, rationales, hedging, and mitigating praise. Prior work informed the initial scheme rather than providing a taxonomy adopted wholesale. We piloted and refined the scheme on 35 comments. Two researchers independently applied the finalized scheme to the full corpus. They then reviewed ambiguous cases and resolved disagreements through discussion. Codes were non-mutually exclusive and described observable characteristics rather than feedback quality. For all 253 textual community contributions, we additionally coded three descriptive feedback attributes: specificity, actionability, and justification. The 19 synthesized comments were further analyzed by tracing their referenced source comments and examining how sources were combined, extended, or reused. The seven replies were included in the corpus-level coding but were not analyzed as a separate quantitative category. Specific comments referred to a concrete aspect, behavior, or situation in the application; actionable comments expressed a change or direction that could be acted upon; and justified comments provided a reason or explanation for an evaluation or suggestion. These attributes were treated as descriptive characteristics rather than measures of feedback quality.

The 19 synthesized comments were additionally analyzed by tracing their referenced source comments and examining how sources were combined, extended, or reused. The seven replies were retained as contextual records but were not analyzed as a separate quantitative category.

Application Revisions and Trajectories. We examined all 60 V0–V1 and V1–V2 transitions by comparing each recorded revision source set with the corresponding before-and-after application versions. Following prior work on feedback-driven application revision [84], we coded each transition in terms of the revision direction expressed by the source, the aspects of the application that changed, observable source–revision correspondence, and source-specific attribution. Comparisons primarily relied on static inspection of HTML, DOM structure, CSS, and JavaScript; runnable versions were additionally inspected when static evidence was insufficient.

For *revision direction*, the recorded source was coded as expressing an identifiable direction, a partially identifiable direction, or no identifiable direction. *Revision aspects* were coded using five non-mutually exclusive categories: *function/feature*, covering additions, removals, or modifications to application capabilities; *interaction/usability*, covering controls, navigation, interaction flows, and other user-facing behaviors; *content/information*, covering displayed text, data, media, or other informational content; *visual style*, covering layout, typography, color, imagery, animation, and other visual presentation; and *conceptual*, covering changes to the application's central purpose, use scenario, or overall framing.

We separately coded *source correspondence*. *Direct correspondence* indicated that the resulting revision visibly followed the direction expressed in the recorded source. *Partial or elaborated correspondence* indicated that the revision implemented only part of that direction or extended it with additional changes. *Indeterminate* was used when the relationship could not be established confidently, and *no observable relevant change* when no corresponding change could be identified. We also checked for materially divergent revisions, in which the resulting change moved against an identifiable source direction.

For source-specific attribution, we recorded whether observable changes could be linked to a single source, involved multiple sources whose individual contributions could not be reliably separated, or lacked sufficient evidence for source-specific attribution. These codes describe observable traceability between recorded inputs and application changes rather than revision quality, effectiveness, Creator acceptance, or fidelity to the Creator's broader intention.

Finally, we compared each application's complete V0–V1–V2 history and assigned one primary trajectory describing its dominant pattern across the two rounds: *structural deepening*, where interactions, pages, or workflows developed into a more complete structure; *cumulative feature expansion*, where functions, content, or states accumulated around the original use; *conceptual clarification or redefinition*, where the application's use scenario, central purpose, or identity became more clearly specified or changed; and *continuous refinement*, where the original concept was retained while existing elements were progressively refined. Two researchers independently coded all 60 version transitions and the 30 complete V0–V1–V2 trajectories, including revision direction, revision aspects, source correspondence, and source-specific attribution. They then cross-reviewed each other's coding in a second pass, and resolved all disagreements through discussion until consensus was reached.

Platform Activity and Post-study Ratings. We descriptively analyzed platform records to characterize exposure to other applications, commenting and liking activity, candidate pools, selection outcomes, and wildcard use. Likes were treated as community-attention signals rather than objective evaluations of comment merit. For participants who contributed ordinary comments in both rounds, we also compared within-participant changes in comment length and selected revision-oriented features. These comparisons were descriptive and were not treated as evidence of learning or causal effects. We summarized the six items adapted from the Exploration, Collaboration, and Expressiveness dimensions of the Creativity Support Index (CSI) at the item level [11]. Participants rated each item on a seven-point Likert scale from 1 (strongly disagree) to 7 (strongly agree). We selected these dimensions because they corresponded most directly to the study's focus on exploring others' applications, participating in shared creation, and expressing ideas through community contributions. We used selected CSI items, thus we report item-level descriptive statistics rather than an overall CSI score.

Interviews. We analyzed the 30 application-centered interviews using reflexive thematic analysis [5]. GPT-5.6 was used to support preliminary coding and the organization of candidate themes using de-identified interview excerpts. Researchers reviewed the source transcripts, revised and consolidated the themes, and retained responsibility for all interpretations and the final analytic account.

6 Findings

Across the study, the 30 participants viewed 4–28 applications created by others (M = 14.77, Md = 13.5), and each application was viewed by 9–25 participants (M = 14.77, Md = 15). They contributed 253 textual contributions: 227 ordinary comments, seven replies, and 19 synthesized comments; 148 appeared in Round 1 and 105 in Round 2. We integrate behavioral, content, artifact, and interview evidence throughout.

6.1 Adapting to a Feedback-to-Revision Structure

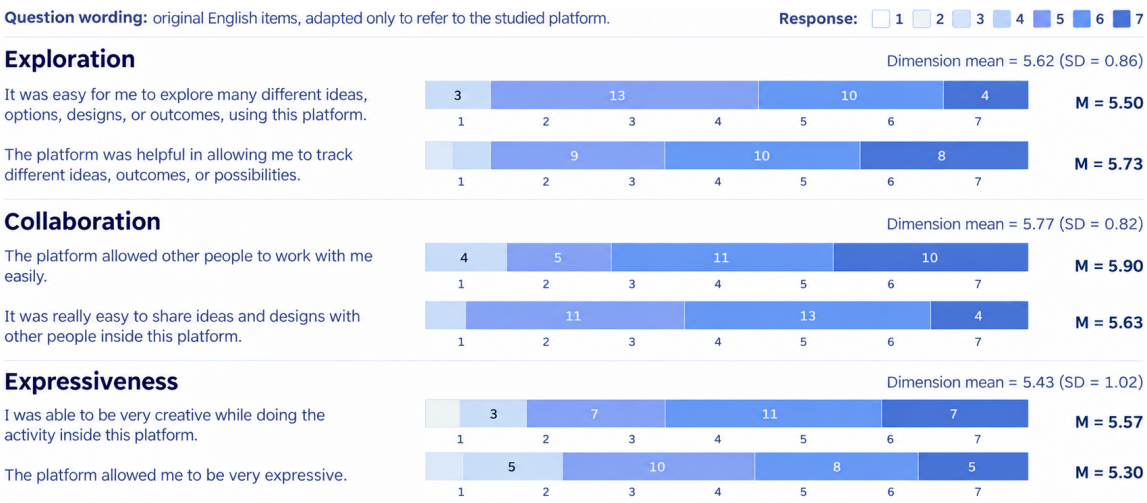

Fig. 7. Post-study ratings for the six adapted CSI items.

As illustrated in Figure 4, the study instantiated a structure in which community feedback could enter AI-mediated application revision and become visible in later versions. In interviews, participants related this experience to feedback practices they had used before the study. These accounts provide participants' own points of reference and do not represent a comparison between experimental conditions. Participants described earlier feedback as a process in which feedback and implementation were separated: a Creator showed a work to others, received suggestions, and decided what to change. C04 described asking a friend to try an application and then improving it personally. C24 emphasized the effort of repeating this process across several people. Here, a community comment could become a source for AI-mediated revision, allowing a Contributor to affect another application without implementing the change personally. Post-study ratings were consistent with this emphasis: enabling others to participate in creation received the highest mean among the six adapted CSI items (M = 5.90, SD = 1.03)(Figure 7), while expressing one's own ideas received the lowest (M = 5.30, SD = 1.15). We treat these ratings descriptively.

Together, these accounts show that participants experienced community feedback as having a more direct material role in development. Figure 8 summarizes how participants adapted to this feedback-to-revision structure.

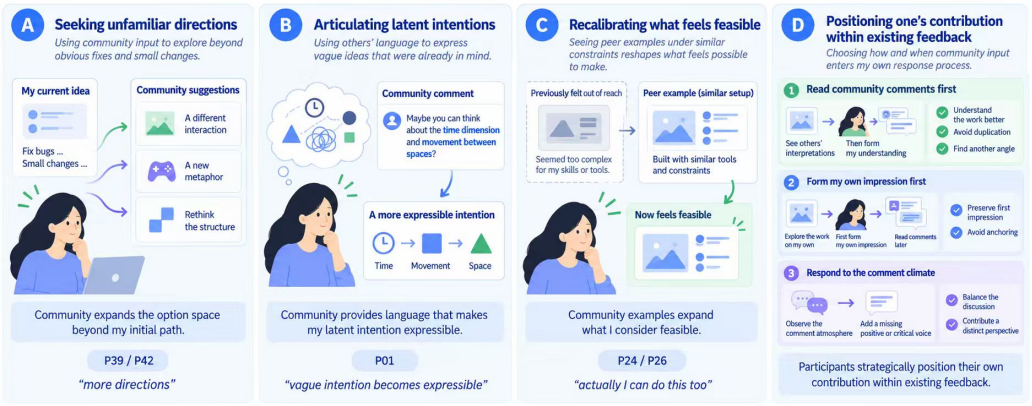

Fig. 8. Observed ways participants adapted to the feedback-to-revision structure. Participants used community input to seek unfamiliar directions, articulate latent intentions, recalibrate what felt feasible, and manage when prior community interpretations entered their creative process.

What participants wanted from the wider community often depended on how much shared context they already had with the people giving feedback. C12 and C26 described friends as better at understanding existing intentions because they shared more context; community members brought fewer shared assumptions and less predictable suggestions. C21 came to the community for "more directions" beyond obvious bugs, and C29 wanted others to introduce a larger change after solo work felt like "the difference between 80 and 81." C10 and C29 also felt that unfamiliar participants could sometimes speak more directly. Participants therefore used familiar contacts to develop known directions and the wider community to expose directions outside the Creator's current frame.

In some cases, the community supplied language or examples that changed what felt available to the Creator. C01 had wanted to develop an application through ideas of time and space but struggled to state that direction; another participant's contribution gave it usable language. Peer applications also changed what seemed feasible. C24 saw an interaction they had assumed would be difficult to produce from a short instruction and concluded, "actually I can do this too." C26 similarly discovered possibilities outside their usual comfort zone. Community participation could therefore articulate latent intentions and recalibrate what participants considered feasible.

Seeing other people's comments introduced another choice: whether to absorb the community's reading of an application before forming one's own response. C12 used earlier comments to understand an unfamiliar application; C21 checked what had already been covered before looking for another angle; and C18 sometimes adjusted their contribution to balance the surrounding comment climate. Other participants delayed reading comments to preserve their first impression. Existing comments therefore influenced both whether participants spoke and what they chose to add, while some participants actively protected an independent first judgment.

6.2 Negotiating Openness and Authorial Boundaries

Participants did not agree on how finished an application needed to be before it was worth opening to others. C11, C13, and C12 considered basic or minimum functionality sufficient once others could meaningfully experience the work, while C19 preferred a more finished state. C25 and C03 found unclear works difficult to comment on, while C26's

relatively complete V0 mainly drew finishing-oriented feedback. Contribution therefore depended on enough form for outsiders to understand the application while important choices still remained open.

Questions of readiness were closely tied to another boundary: which parts of an application were open to change. As Figure 9 shows, function or feature changes appeared in 53 of the 60 transitions (88.3%), interaction or usability changes in 50 (83.3%), content or information changes in 45 (75.0%), visual-style changes in 37 (61.7%), and broader conceptual changes in 14 (23.3%). C20 wanted to preserve core functions while allowing mood-related features, timing, and front-end details to develop; C03 welcomed changes to color, scale, or layout while preserving the core concept; C08 protected the "main direction" while allowing substantial interface change; and C30 separated smaller optimization from larger decisions they preferred to make themselves. These patterns show that openness was often selective within the same artifact, with broader conceptual movement carrying a different weight from local functional or visual change.

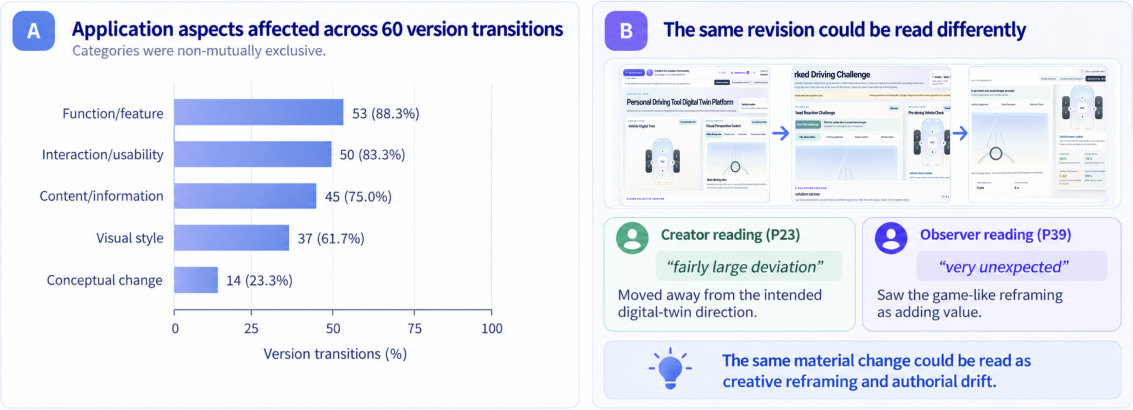

Fig. 9. Application changes and divergent interpretations of revision outcomes. (A) Application aspects affected across 60 version transitions; categories were non-mutually exclusive. (B) An example in which the same revision was interpreted differently by the creator and another participant, illustrating how an implemented change could be read as either deviation from the original direction or a valuable reframing.

Those boundaries were sometimes easiest to recognize after a revision had already crossed them. C10's application was intended as a low-purpose environment for "zoning out," yet a community revision moved it toward ADHD training; seeing that version helped C10 state more clearly that the work should remain free of a training purpose. C10 still found the alternative interesting and remained curious about where the community might take the work. The strongest contrast came from the same revision history: C23 saw the shift of a car-oriented digital-twin application toward a driving-test game as a "fairly large deviation," while C21 saw the same trajectory as "very unexpected" and more valuable than the original. The same revision could therefore be valuable to one participant while crossing the Creator's intended boundary for another.

For some participants, these boundaries shaped publication decisions before any feedback was given. C01 and C03 showed stronger concern for preserving direction when speaking as Creators than when proposing changes to someone else's work. C09 also said that knowing a space was built around continued co-creation would affect what they chose to publish: a base idea could be released earlier when change was expected, while strongly personal work would be kept outside such a setting. Authorial boundaries therefore shaped both how participants judged revisions and which artifacts they considered suitable for community development.

6.3 Participants Treated Commenting as More Consequential

The comments themselves rarely fit a single social or revision-oriented role. Figure 10 summarizes the characteristics of the 253 text contributions. In Figure 10A, direct recommendations appeared in 185 comments (73.1%), evaluative judgments in 156 (61.7%), social or phatic responses in 66 (26.1%), process diagnoses in 33 (13.0%), and comparisons or references in 30 (11.9%). Revision-oriented forms were also mixed (Figure 10B): 148 comments (58.5%) specified a desired direction, 116 (45.8%) identified an issue or notable feature, and 61 (24.1%) proposed a concrete solution or implementation route. As Figure 10C shows, hedging appeared in 135 comments (53.4%) and mitigating praise in 58 (22.9%); evaluation and recommendation co-occurred in 92 comments, while 89 combined hedging with a suggestion. Finally, Figure 10D shows that 205 contributions (81.0%) were specific, 171 (67.6%) were actionable, and 133 (52.6%) were justified. Together, these patterns show that comments could provide concrete directions for revision while still carrying evaluation, caution, and ordinary social meaning.

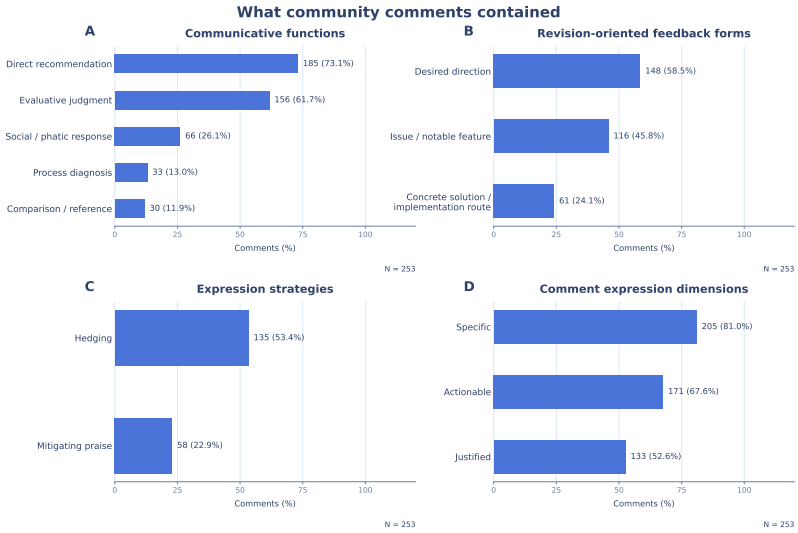

Fig. 10. Characteristics of the 253 community text contributions. (A) Communicative functions. (B) Revision-oriented feedback forms. (C) Expression strategies. (D) Comment expression dimensions. Categories were coded using multi-label schemes, so percentages within each panel do not necessarily sum to 100%.

Knowing that a comment might be executed did not lead participants toward a single writing style. C09 still wrote "for the Creator to read" and wanted comments to retain the natural tone of social feedback. C26, C02, and C21 paid more attention to how wording might be passed to AI, using clearer or more structured descriptions when a change felt important. C27 moved in another direction: they had a concrete animation idea but deliberately described it vaguely because an exact but wrong reference might make AI "mess it up." C30 described another form of restraint, avoiding some larger ideas because they were already thinking about whether AI could implement them well. Across rounds, actionability showed the clearest directional increase among the 16 participants who contributed ordinary comments in both rounds: 11 increased actionable comments, nine increased specificity, nine increased median comment length, and eight increased justification. These patterns were consistent with participants treating wording as more consequential

when comments could enter revision, while managing that possibility through specificity, ambiguity, restraint, or continued human-facing expression.

This ambiguity became most visible when ordinary social language entered the revision pathway. In one case, C13 wrote "Set off fireworks for myself," using the phrase as a metaphorical expression of feeling. After the comment was selected for revision, the generated application treated the phrase more literally and turned it into an interactive feature (Figure 11). The result surprised the participant and suggested another way the application could work. Once comments entered revision, the boundary between social expression and executable instruction could become unstable.

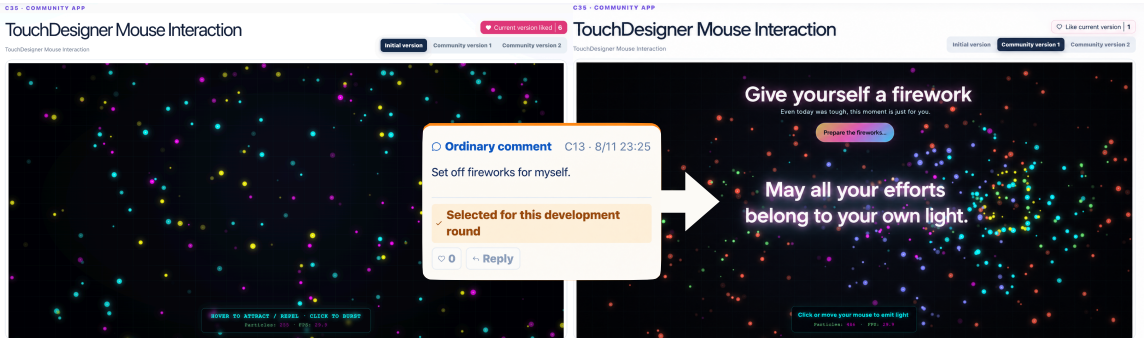

Fig. 11. A social expression became an executable interaction after entering revision. C13's comment, "Set off fireworks for myself," was selected for development and materialized in the subsequent application version as an interactive "Give yourself a firework" feature.

Once a comment produced a visible change, participants sometimes reconsidered what they had actually said. C17 immediately wondered whether they had "said something random," because a casual remark had acquired a visible consequence in someone else's artifact. C28, however, described affective and experiential comments as useful even when they offered no clear revision direction, because they could tell the Creator that an experience worked and motivate further creation. Execution therefore changed the consequence of some comments without replacing the social value of comments that remained evaluative, supportive, or expressive.

6.4 Selection Bounded Materialization, Not Influence

Only a small part of the available feedback could enter any one revision. Across two rounds, the platform made 60 selections, one per application per round, and candidate pools contained 1–9 eligible comments ($M = 3.97$, $Md = 4$). Twenty-five pools contained no liked comments and 36 contained ties for the highest like count. Of the 60 platform-selected comments, 50 were in the highest-liked tier, including 15 that were uniquely highest-liked, while 10 fell below the highest-liked tier.

Creators also used the Wildcard in 25 transitions, adding 24 ordinary comments and one reply as additional revision inputs. All 25 Wildcard-selected contributions were specific, 23 were actionable, and 19 were justified. Among the 16 Wildcard contributions from app-rounds with at least two likes in total, 13 (81.3%) were highest-liked or tied for the highest like count, one had likes but was not highest-liked, and two had zero likes.

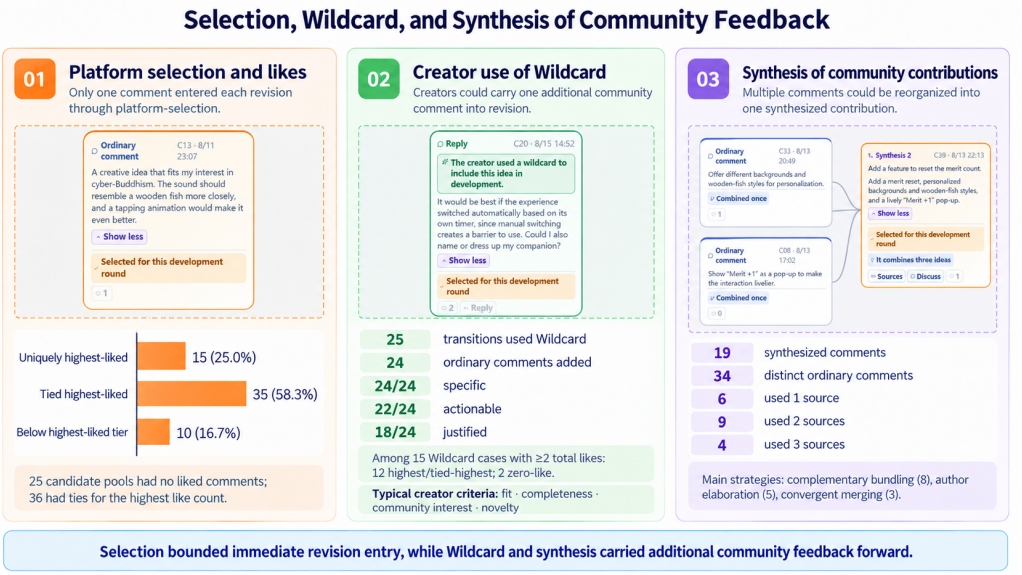

Fig. 12. Selection, Wildcard use, and synthesis of community feedback. Panel A shows how platform-selected comments related to like counts. Panel B summarizes Creator use of the Wildcard to carry an additional community contribution into revision. Panel C shows how existing comments were reorganized into synthesized contributions.

The 19 synthesized comments drew on 36 direct-source instances representing 34 distinct contributions. Six syntheses used one source, nine used two, and four used three. Complementary bundling was most common, followed by author elaboration and convergent merging. Figure 12 summarizes these three pathways through which community feedback was selected, carried forward, and reorganized.

That constraint quickly exposed a mismatch between what was selected and what participants considered valuable. C05 felt awkward when a casual comment was selected over a more careful one, and C14 worried that a weak random choice could use up an iteration opportunity. Likes created a related mismatch. C24 used likes for agreement and for ideas that felt interesting or surprising; C21 said they had liked every comment under their own application; C07 at one point used broad liking to avoid offending anyone; and C18 noted that high engagement could also follow controversy. A like therefore carried several social meanings while also affecting execution probability, so popularity could not be read as a single measure of implementation preference.

Creator intervention introduced another layer of judgment, and the criteria varied across participants. C07 favored a detailed suggestion that could make the application more complete, C25 prioritized alignment with their own direction, and C30 attended to visible community interest. C02 valued contributions that introduced promising directions they had not considered before. Creator intervention therefore reflected several criteria: fit, completeness, collective interest, and novelty.

Comments could remain useful even when they were never materialized, and popularity did not determine their longer-term value. C21 said obvious problems such as a broken button were often things a Creator already knew, while a less popular comment could introduce a direction the Creator had never considered. Unselected comments could remain active as well. C14 described an unselected idea about audience reviews as a direction worth developing later; C16 said an unselected suggestion to add a small monk character could still become part of future work; and C18

separated an immediately feasible revision from a larger community feature that belonged to a longer horizon. C12 also described a larger pool of ideas as a source of decision burden when suggestions pointed in different directions. Selection therefore bounded what entered the current revision, while influence continued through comments that were remembered, combined, postponed, or weighed against competing ideas.

Materialization also carried a social meaning that the revision mechanism alone could not determine. C24 described partial ownership after seeing a contribution become visible, while C17 and C27 described achievement or involvement. C28 drew a clear line between system selection and Creator recognition: being deliberately chosen by a Creator felt more meaningful because another person had seen the contribution and wanted to use it. This distinction matters because the same comment could have a material effect under system selection or Creator intervention while producing a different sense of collaboration for the Contributor. C27 and C01 also described thresholds for authorship, with small or common suggestions carrying less ownership than unique changes affecting the framework or core logic. Participants applied a similar threshold to attribution: credit mattered more when a contribution felt distinctive or central to the resulting artifact, while small or widely available suggestions did not always create the same expectation of being named. Material consequence, social recognition, perceived authorship, and attribution therefore remained separate dimensions of contribution.

6.5 Materialized Versions as Generative and Contrastive References

At the artifact level, most revisions remained visibly connected to their recorded sources. 33 of the 60 transitions (55.0%) showed direct observable correspondence and 21 (35.0%) showed partial or elaborated correspondence; five were indeterminate and one showed no observable relevant change. The recorded source contained an identifiable revision direction in 50 transitions, a partially identifiable direction in five, and no identifiable direction in five. None of the transitions was coded as materially divergent from the recorded source direction. Single-source attribution was observable in 29 transitions, while 25 involved multiple sources whose contributions could not be reliably separated. In the remaining six transitions, source-specific attribution was not possible. As shown in Figure 13, most generated changes remained visibly connected to their recorded revision sources.

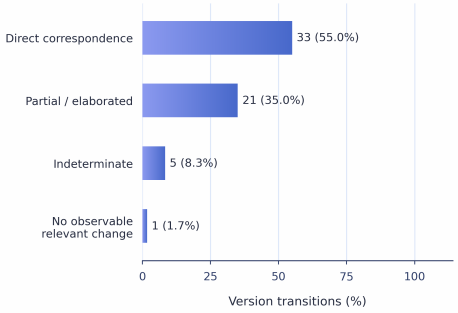

Fig. 13. Observed correspondence between recorded revision sources and the resulting application changes across 60 version transitions.

These results establish that most generated changes were visibly related to their recorded sources, while still leaving room for consequences that the source text had not fully specified. This coding assessed correspondence to the recorded

revision source, not fidelity to the Creator's broader intention. A source-consistent revision could therefore still feel like a major authorial drift or reveal consequences that the comment had not specified.

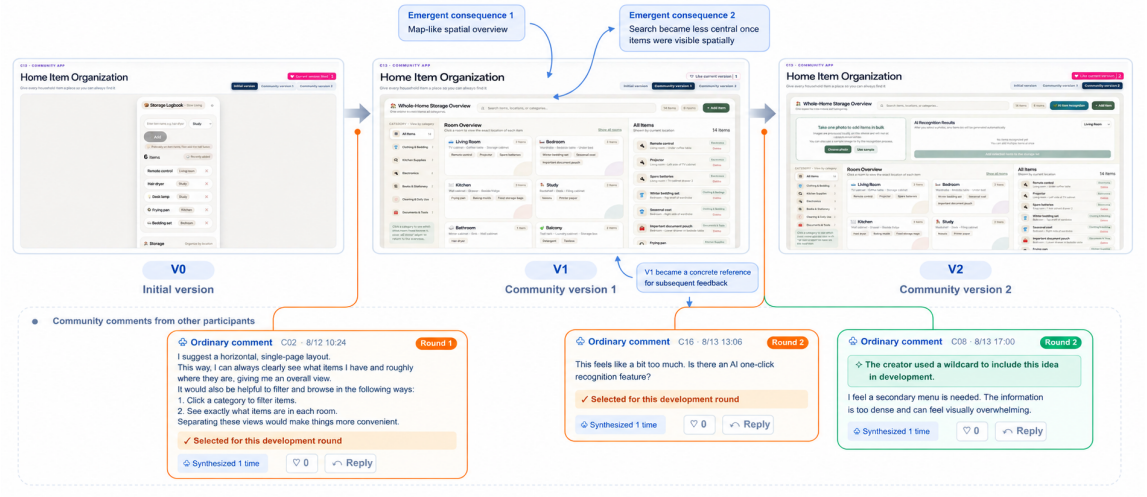

Fig. 14. Application C13 across V0, V1, and V2. A Round 1 community suggestion for a horizontal, single-page layout was selected and materialized in V1. The realized version produced a map-like spatial overview and made search feel less central once items became visible by location. V1 then became a concrete reference for subsequent Round 2 feedback, including one platform-selected comment and one Creator-added Wildcard contribution.

What source correspondence did not capture was what became apparent only after a suggestion took concrete form. C13 provided a clear example, illustrated in Figure 14. A selected comment proposed turning a vertically organized application into a horizontal layout. Before seeing the revision, C13 mainly understood this as adapting the interface to a wider screen. The generated version created an unexpected map-like spatial overview, and C13 said that "seeing the result makes it easier to get inspiration." The new layout also changed how they thought about another feature: once objects could be located at a glance, the search function seemed less central. The case shows how a materialized version could reveal interactions between a suggested direction and the existing artifact that were difficult to anticipate from language alone.

Other cases followed a more indirect route. C28's comment was interpreted differently from what they meant, and the resulting lighting effect also fell outside the Creator's original intention. The visible effect still led the Creator toward another firework-like visual idea. C28 described the route as a "roundabout" connection. The contribution became useful through an implementation that neither side had asked for, and the unexpected result then became material for another idea. Concrete versions could also work through rejection: C10's ADHD-oriented revision made the boundary of the zoning-out application clearer, while C30 described disliked AI-generated outcomes as helping them identify what they actually wanted. These cases show two ways a version could support further thinking: an unexpected consequence could become new material, and an unwanted consequence could make a boundary more explicit.

Across the full V0–V1–V2 trajectories, these effects accumulated unevenly: iteration could add one element and lose another. Across complete trajectories, nine applications showed structural deepening, eight cumulative feature

expansion, seven conceptual clarification or redefinition, and six continuous refinement. C26 described an earlier memory-card interaction they found especially clear and effective, yet that effect disappeared in a later version. C14 noted that once an early comment had been materialized, later work proceeded from that version, so Round 2 participants responded to a changed artifact. C05, C22, C10, and C04 all linked this uncertainty to the value of keeping earlier versions available or being able to return to them. Materialized versions therefore functioned as working references across time: they could expose new consequences, clarify boundaries, shape later feedback, and preserve alternatives that might otherwise disappear.

7 Discussion

Once community feedback can enter AI-mediated revision, it no longer sits outside the act of making. A comment can alter an executable application, produce unanticipated consequences, and change what later participants encounter. In our study, this shifted who could influence development, how people wrote feedback, and how each generated version shaped what happened next.

7.1 From Feedback to Consequential Participation

Most work on collaborative natural-language programming starts with people who are already part of development. CoPrompt supports collaborators in sharing and referring to prompts [19]; work on generative-AI prototyping examines coordination inside software teams [72]; and recent Vibe Coding research describes collaboration around prototyping, evaluation, trust, and responsibility in product teams [44]. Participatory Vibe Coding widens that circle by showing how domain participants can translate situated knowledge into specifications [35]. Community feedback begins from a different position. Contributors can evaluate, diagnose, suggest, or reframe a work without taking responsibility for implementation, while creators interpret and prioritize what follows [20, 21, 47]. In CoVibe, these relations no longer remained separate: a Contributor could affect another person's runnable application through a comment while staying outside implementation. We use *consequential participation* to describe this intermediate position, where a contribution has a system-supported route to material change without giving its author control over the full path from expression to implementation.

That path was split across several actors and mechanisms. Contributors decided what to say; likes altered selection probability; the weighted lottery allocated one platform-selected revision source per application and round; a Creator could add one existing contribution through the Wildcard across the two rounds; and the model operationalized the resulting source set. Mixed-initiative creativity has long treated agency as distributed across human and computational actions [17], while recent Vibe Coding work describes intention, authorship, and responsibility as entangled across people and generated code [75]. A broader review similarly argues that human–AI agency is better understood through concrete control mechanisms than assigned wholesale to either side [91]. CoVibe adds a community layer: control was distributed across Contributors, Creators, community attention, selection, and the model.

The distinction sharpened when material influence was compared with recognition and authorship. Participants treated platform selection differently from deliberate Creator nomination even when both produced visible revisions, and they made stronger authorship claims for distinctive changes to an application's framework or core logic than for small or common suggestions. This is consistent with remix research showing that automatic provenance and human-given credit carry different social meanings [54], and with work separating contribution integration from attribution [49]. Human–AI co-creation research likewise finds that credit judgments vary with contribution type, amount, and initiative [28]. Selection determined what could be materialized in the current round, not the full value

of feedback. Unselected comments could still be remembered, combined, postponed, or used to frame later decisions. Creator agency also began before any comment was written, through choices about what to publish, how complete it should be, and which parts remained open to change. That pattern echoes work showing that feedback seeking depends on perceived readiness and artifact maturity [39, 42]. Seen this way, participation is layered: proposing a change, having it materialized, receiving recognition, and claiming authorship may coincide, but they often do not.

7.2 When Social Language Becomes Development Material

Natural-language programming usually treats language as a way to express development intent, even when users revise that intent after inspecting generated outputs [3, 52, 65]. Community comments carry a broader social load: a comment can evaluate, recommend, hedge, encourage, diagnose, or signal a relationship to the work [20, 40, 56]. In CoVibe, those ordinary communicative functions entered a revision pathway without disappearing. The 253 contributions did not become prompt-like instructions: 61.7% contained evaluative judgments, 53.4% used hedging, and 26.1% included social or phatic responses, even though 81.0% were specific and 67.6% actionable. The change was not that comments became prompts. They became development material while still behaving like social communication.

C13's "Set off fireworks for myself" makes this tension concrete. The phrase was written as a metaphorical expression of feeling, then materialized as an interactive feature. Nothing in the comment required that reading, yet the executable version made one interpretation visible and consequential. Feedback research shows that recipients make sense of critique through their goals, expertise, and reading of context [21, 40]; Vibe Coding research similarly shows that generated outputs can reshape the intent that entered the system [52, 75]. Here, those processes met visibly: AI-mediated revision turned one interpretation of socially situated language into a runnable state that the Creator and wider community then had to respond to.

Participants did not respond uniformly. Some wrote more clearly when they cared about a particular change. Others kept comments deliberately vague because an overly precise reference could push the model in the wrong direction, filtered out ideas they considered difficult to implement, or continued writing primarily for the Creator. Those choices matter because feedback cannot be reduced to specification. Positive affect can improve how critique is received [56], and feedback exchanges also sustain motivation, interpretation, and social connection [20]. Treating every comment as a latent instruction would flatten those functions. Collaborative Vibe Coding instead gives feedback a dual audience: comments remain part of a human exchange while also carrying the possibility of executable consequence.

7.3 Executable Versions Reconfigure Subsequent Collaboration

Generated outputs in code-generation and generative-AI prototyping are rarely endpoints; people inspect them, compare them with what they intended, and decide what to repair or pursue next [3, 72]. Feedback-driven design likewise uses intermediate artifacts and revision histories to make alternatives inspectable [26, 71]. In our study, an intermediate version also became the object of later community participation. A materialized version did not only answer the contribution that preceded it; it became the application that later participants saw. Once V0 became V1, Round 2 feedback was produced against a changed artifact. One AI-mediated interpretation therefore altered the conditions of the next round of collaboration.

Source correspondence and authorial alignment are therefore different questions. Fifty-four of the 60 transitions showed direct or partial/elaborated correspondence with their recorded sources, yet a source-consistent revision could still create interactions among existing features, visual choices, or conceptual implications that the comment had never specified. Materialization made one interpretation inspectable. When participants liked an unexpected consequence,

the version became material for further development; when they disliked it, the version could still show more clearly what should be preserved or rejected. An unwanted result could still be productive by exposing an authorial boundary that had previously remained implicit. This develops accounts of Vibe Coding as iterative intent mediation [52, 75]: in a community setting, the generated artifact mediates not only between a person and a model, but among participants across revision rounds.

Version history also mattered because iteration could erase as well as add. Participants noticed useful interactions disappearing in later revisions and wanted access to prior states. Research on change tracking and collaborative histories shows how preserved alternatives support reflection and interpretation [26, 71, 79]. In collaborative Vibe Coding, those histories also record how community language was operationalized and what later participants actually encountered. The V0–V1–V2 sequence is more than provenance: it is a shared interpretive record of proposed directions, materialized consequences, retained or rejected possibilities, and later feedback. Revision changes the artifact, and the changed artifact changes what the community can see and respond to next.

7.4 Design Implications

7.4.1 Make selection and execution pathways legible. A collaborative Vibe Coding system should make the route from comment to revision understandable without presenting selection as comment quality. Users should be able to tell when a comment is eligible for revision, how likes affect selection probability, whether it entered through platform selection or Creator intervention, and which changes trace back to it. Likes in our study carried several social meanings, while selection allocated only a limited materialization opportunity. Traceability should explain how influence was allocated, not imply Creator endorsement or community consensus. Work on creative ancestry similarly shows that visible contribution histories can shape perceived fairness in collaborative work [57].

7.4.2 Separate materialization, recognition, and attribution. These relations should not collapse into one interface signal. A comment may be materialized without Creator endorsement; a Creator may value an unselected comment; and a visible change may warrant provenance without automatically creating an authorship claim. System-generated provenance can document what entered a revision, while explicit acknowledgment can show that another person noticed and valued the contribution. This is especially useful for ideas that matter but are never selected. Prior work shows that appreciation and attention can sustain contribution in online knowledge networks [73], while automatic attribution does not substitute for socially meaningful credit [28, 54]. Keeping these relations separate avoids equating implementation with recognition or non-selection with irrelevance.

7.4.3 Preserve versions as resources for continued participation. Versioning should help people continue the work, not only document the past. Systems should preserve links among source comments, selection events, generated revisions, and earlier runnable states so participants can compare interpretations, recover useful effects that disappeared, and understand how the current artifact came to be. Earlier versions can remain active resources for deciding what to continue, reverse, or reopen. Provenance is therefore not simply a record of who changed what; it helps the community reason about what the application could become next.

8 Limitations

This study has several limitations. First, our deployment was conducted over only 8 days and captured early-stage collaborative practices. Participants experienced a limited number of interactions and revision cycles. We could examine

how community feedback entered AI-mediated revision, but we lack data on how collaboration may develop in longer-running communities. Future studies could investigate sustained Vibe Coding communities over longer periods.

Second, participants engaged with the platform to different extents. Although all participants were encouraged to experience multiple applications and contribute feedback, the depth of browsing, commenting, and interaction varied across individuals and applications. Future work could further examine how different participation patterns shape community-based creative processes.

Third, the revision process was influenced by the specific AI model and implementation used in this study. CoVibe used GPT-5.6 Sol through Codex, and other models or system configurations may interpret and implement the same community feedback differently. Future research could explore how variations in AI capabilities and interaction designs affect feedback-driven application evolution.

Fourth, participants were invited to join the platform and provide feedback, rather than choosing to use the platform on their own. Their motivations and levels of engagement may therefore differ from those of users who voluntarily use similar platforms, which may limit the generalizability of our findings to naturally occurring platform use.

9 Conclusion

Vibe Coding creates a pathway through which community feedback can enter AI-mediated application revision while remaining part of a social exchange. Through formative interviews with 25 participants, we identified challenges in organizing how external contributions could enter revision under limited revision opportunities while remaining traceable across application versions. We translated these challenges into design goals and developed CoVibe as a research platform that makes this feedback-to-revision structure concrete and observable. We then used CoVibe in an eight-day deployment with 30 participants to investigate how people participated in and interpreted this structure. Our findings show that creators negotiated what was open to change, while contributors balanced social communication with the possibility that their comments could enter revision. Selection bounded which contributions were materialized without bounding their broader influence, and materialized versions made particular interpretations of feedback concrete, revealing consequences that were difficult to anticipate from language alone. Together, this work contributes a formative understanding of collaborative Vibe Coding and a research platform that operationalizes this feedback-to-revision structure, as well as an empirical account of how participants engage with and interpret the structure across evolving application versions. Our findings additionally inform how such collaborative workflows may be organized and designed.

References

[1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 3, 13 pages. doi:10.1145/3290605.3300233

[2] Ant Group. 2025. LingGuang: Multimodal AI Assistant and Flash-App Creation Platform. Mobile application. https://apps.apple.com/cn/app/id6751496092 Accessed: 2026-09-10.

[3] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1, Article 78 (apr 2023), 27 pages. doi:10.1145/3586030

[4] Karim Benharrak, Tim Zindulka, Florian Lehmann, Hendrik Heuer, and Daniel Buschek. 2024. Writer-Defined AI Personas for On-Demand Feedback Generation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 1049, 18 pages. doi:10.1145/3613904.3642406

[5] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa

[6] Bill Buxton. 2007. *Sketching User Experiences: Getting the Design Right and the Right Design.* Morgan Kaufmann, San Francisco, CA, USA. doi:10.1016/B978-0-12-374037-3.X5043-3

[7] Julia Cambre and Scott Klemmer. 2017. Standing on the Shoulders of Peers: Tournament-Style Remixing in Project Courses. In *Companion of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing* (Portland, OR, USA) *(CSCW '17 Companion).* Association for Computing Machinery, New York, NY, USA, 155–158. doi:10.1145/3022198.3026353

[8] Gengchen Cao, Tianke He, Yixuan Liu, and Ray LC. 2026. Audience in the Loop: Viewer Feedback-Driven Content Creation in Micro-Drama Production on Social Media. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems* (Barcelona, Spain) *(CHI '26).* Association for Computing Machinery, New York, NY, USA, Article 1287, 25 pages. doi:10.1145/3772318.3790592

[9] Ruijia Cheng and Jenna Frens. 2022. Feedback Exchange and Online Affinity: A Case Study of Online Fanfiction Writers. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2, Article 402 (2022), 29 pages. doi:10.1145/3555127

[10] Ruijia Cheng, Ziwen Zeng, Maysnow Liu, and Steven P. Dow. 2020. Critique Me: Exploring How Creators Publicly Request Feedback in an Online Critique Community. *Proceedings of the ACM on Human-Computer Interaction* 4, CSCW2, Article 161 (oct 2020), 24 pages. doi:10.1145/3415232

[11] Erin Cherry and Celine Latulipe. 2014. Quantifying the Creativity Support of Digital Tools through the Creativity Support Index. *ACM Transactions on Computer-Human Interaction* 21, 4, Article 21 (2014), 25 pages. doi:10.1145/2617588

[12] Yoonseo Choi, Eun Jeong Kang, Seulgi Choi, Min Kyung Lee, and Juho Kim. 2025. Proxona: Supporting Creators' Sensemaking and Ideation with LLM-Powered Audience Personas. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems.* Association for Computing Machinery, New York, NY, USA, Article 149, 32 pages. doi:10.1145/3706598.3714034

[13] Yoonseo Choi, Toni-Jan Keith Palma Monserrat, Jeongeon Park, Hyungyu Shin, Nyoungwoo Lee, and Juho Kim. 2020. ProtoChat: Supporting the Conversation Design Process with Crowd Feedback. *Proceedings of the ACM on Human-Computer Interaction* 4, CSCW3, Article 225 (2020), 27 pages. doi:10.1145/3432924

[14] Yi-Hung Chou, Boyuan Jiang, Yi Wen Chen, Mingyue Weng, Victoria Jackson, Thomas Zimmermann, and James A. Jones. 2026. Building Software by Rolling the Dice: A Qualitative Study of Vibe Coding. *Proceedings of the ACM on Software Engineering* 3, FSE, Article FSE077 (jul 2026), 23 pages. doi:10.1145/3797105

[15] Patrick A. Crain and Brian P. Bailey. 2022. Easier Said or Easier Done? Exploring the Relative Merits of Common Feedback Presentations. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW1, Article 86 (2022), 19 pages. doi:10.1145/3512933

[16] Sayamindu Dasgupta, William Hale, Andres Monroy-Hernandez, and Benjamin Mako Hill. 2016. Remixing as a Pathway to Computational Thinking. In *Proceedings of the 19th ACM Conference on Computer-Supported Cooperative Work and Social Computing* (San Francisco, California, USA) *(CSCW '16).* Association for Computing Machinery, New York, NY, USA, 1438–1449. doi:10.1145/2818048.2819984

[17] Sebastian Deterding, Jonathan Hook, Rebecca Fiebrink, Marco Gillies, Jeremy Gow, Memo Akten, Gillian Smith, Antonios Liapis, and Kate Compton. 2017. Mixed-Initiative Creative Interfaces. In *Proceedings of the 2017 CHI Conference Extended Abstracts on Human Factors in Computing Systems.* Association for Computing Machinery, New York, NY, USA, 628–635. doi:10.1145/3027063.3027072

[18] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2026. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook – a Grey Literature Review. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice* (Rio de Janeiro, Brazil) *(ICSE-SEIP '26).* Association for Computing Machinery, New York, NY, USA, 212–223. doi:10.1145/3786583.3786866

[19] Li Feng, Ryan Yen, Yuzhe You, Mingming Fan, Jian Zhao, and Zhicong Lu. 2024. CoPrompt: Supporting Prompt Sharing and Referring in Collaborative Natural Language Programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24).* Association for Computing Machinery, New York, NY, USA, Article 934, 21 pages. doi:10.1145/3613904.3642212

[20] Eureka Foong, Steven P. Dow, Brian P. Bailey, and Elizabeth M. Gerber. 2017. Online Feedback Exchange: A Framework for Understanding the Socio-Psychological Factors. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems.* Association for Computing Machinery, New York, NY, USA, 4454–4467. doi:10.1145/3025453.3025791

[21] Eureka Foong, Darren Gergle, and Elizabeth M. Gerber. 2017. Novice and Expert Sensemaking of Crowdsourced Design Feedback. *Proceedings of the ACM on Human-Computer Interaction* 1, CSCW, Article 45 (2017), 18 pages. doi:10.1145/3134680

[22] Jonas Frich, Lindsay MacDonald Vermeulen, Christian Remy, Michael Mose Biskjaer, and Peter Dalsgaard. 2019. Mapping the Landscape of Creativity Support Tools in HCI. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems.* Association for Computing Machinery, New York, NY, USA, Article 389, 18 pages. doi:10.1145/3290605.3300619

[23] Kiev Gama, Filipe Calegario, Victoria Jackson, Alexander Nolte, Luiz Augusto Morais, and Vinicius Cardoso Garcia. 2026. "Can You Feel the Vibes?": An Exploration of Novice Programmer Engagement with Vibe Coding. In *2026 IEEE/ACM 48th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET) (ICSE-SEET '26).* Association for Computing Machinery, New York, NY, USA, 485–496. doi:10.1145/3786580.3786992

[24] Michael D. Greenberg, Matthew W. Easterday, and Elizabeth M. Gerber. 2015. Critiki: A Scaffolded Approach to Gathering Design Feedback from Paid Crowdworkers. In *Proceedings of the 2015 ACM SIGCHI Conference on Creativity and Cognition.* Association for Computing Machinery, New York, NY, USA, 235–244. doi:10.1145/2757226.2757249

[25] Qingyu Guo, Chao Zhang, Hanfang Lyu, Zhenhui Peng, and Xiaojuan Ma. 2023. What Makes Creators Engage with Online Critiques? Understanding the Role of Artifacts' Creation Stage, Characteristics of Community Comments, and Their Interactions. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems.* Association for Computing Machinery, New York, NY, USA, Article 556, 17 pages. doi:10.1145/3544548.3581054

[26] Bjorn Hartmann, Sean Follmer, Antonio Ricciardi, Timothy Cardenas, and Scott R. Klemmer. 2010. d.note: Revising User Interfaces through Change Tracking, Annotations, and Alternatives. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Atlanta, Georgia, USA) *(CHI '10)*. Association for Computing Machinery, New York, NY, USA, 493–502. doi:10.1145/1753326.1753400

[27] Sarah Harvey. 2014. Creative Synthesis: Exploring the Process of Extraordinary Group Creativity. *Academy of Management Review* 39, 3 (2014), 324–343. doi:10.5465/amr.2012.0224

[28] Jessica He, Stephanie Houde, and Justin D. Weisz. 2025. Which Contributions Deserve Credit? Perceptions of Attribution in Human-AI Co-Creation. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 540, 18 pages. doi:10.1145/3706598.3713522

[29] Benjamin Mako Hill and Andres Monroy-Hernandez. 2013. The Remixing Dilemma: The Trade-Off Between Generativity and Originality. *American Behavioral Scientist* 57, 5 (may 2013), 643–663. doi:10.1177/0002764212469359

[30] Benjamin Mako Hill, Andrés Monroy-Hernández, and Kristina R. Olson. 2010. Responses to Remixing on a Social Media Sharing Website. In *Proceedings of the Fourth International AAAI Conference on Weblogs and Social Media*. AAAI Press, Washington, DC, USA, 74–81.

[31] Hsiu-Fang Hsieh and Sarah E. Shannon. 2005. Three Approaches to Qualitative Content Analysis. *Qualitative Health Research* 15, 9 (2005), 1277–1288. doi:10.1177/1049732305276687

[32] Yu-Kai Hung, Yun-Chien Huang, Ting-Yu Su, Yen-Ting Lin, Lung-Pan Cheng, Bryan Wang, and Shao-Hua Sun. 2025. SimTube: Simulating Audience Feedback on Videos Using Generative AI and User Personas. In *Proceedings of the 30th International Conference on Intelligent User Interfaces (IUI '25)*. Association for Computing Machinery, New York, NY, USA, 1256–1271. doi:10.1145/3708359.3712146

[33] Andrej Karpathy. 2025. There's a New Kind of Coding I Call "Vibe Coding". Post on X. https://x.com/karpathy/status/1886192184808149383 Accessed: 2026-09-10.

[34] Joy Kim, Justin Cheng, and Michael S. Bernstein. 2014. Ensemble: Exploring Complementary Strengths of Leaders and Crowds in Creative Collaboration. In *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work and Social Computing* (Baltimore, Maryland, USA) *(CSCW '14)*. Association for Computing Machinery, New York, NY, USA, 745–755. doi:10.1145/2531602.2531638

[35] Junghwan Kim, Kieun Park, Jeongwoo Ryu, Hyungwoo Song, and Bongwon Suh. 2026. When Nurses Meet Vibe-Coding: Shifting from Problem Talk to Situated Specification in Participatory Design. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems* (Barcelona, Spain) *(CHI EA '26)*. Association for Computing Machinery, New York, NY, USA, Article 761, 7 pages. doi:10.1145/3772363.3798470

[36] Joy Kim, Sarah Sterman, Allegra Argent Beal Cohen, and Michael S. Bernstein. 2017. Mechanical Novel: Crowdsourcing Complex Work through Reflection and Revision. In *Proceedings of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing* (Portland, Oregon, USA) *(CSCW '17)*. Association for Computing Machinery, New York, NY, USA, 233–245. doi:10.1145/2998181.2998196

[37] Amy J. Ko, Robin Abraham, Laura Beckwith, Alan F. Blackwell, Margaret Burnett, Martin Erwig, Christopher Scaffidi, Joseph Lawrance, Henry Lieberman, Brad A. Myers, Mary Beth Rosson, Gregg Rothermel, Mary Shaw, and Susan Wiedenbeck. 2011. The State of the Art in End-User Software Engineering. *Comput. Surveys* 43, 3, Article 21 (2011), 44 pages. doi:10.1145/1922649.1922658

[38] Charlotte Kobiella, Daniela Breidenstein, and Albrecht Schmidt. 2026. From Throw-Away to Takeaway: How GenAI and Vibe Coding Accelerate Prototyping Across Technical Skill Levels. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems* (Barcelona, Spain) *(CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 688, 23 pages. doi:10.1145/3772318.3790757

[39] Yasmine Kotturi and McKayla Kingston. 2019. Why do Designers in the "Wild" Wait to Seek Feedback until Later in their Design Process?. In *Proceedings of the 2019 on Creativity and Cognition* (San Diego, CA, USA) *(C&C '19)*. Association for Computing Machinery, New York, NY, USA, 541–546. doi:10.1145/3325480.3326580

[40] Yubo Kou and Colin M. Gray. 2017. Supporting Distributed Critique through Interpretation and Sense-Making in an Online Creative Community. *Proceedings of the ACM on Human-Computer Interaction* 1, CSCW, Article 60 (nov 2017), 18 pages. doi:10.1145/3134695

[41] Markus Krause, Thomas Garncarz, Jiaojiao Song, Elizabeth M. Gerber, Brian P. Bailey, and Steven P. Dow. 2017. Critique Style Guide: Improving Crowdsourced Design Feedback with a Natural Language Model. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 4627–4639. doi:10.1145/3025453.3025883

[42] Sneha R. Krishna Kumaran, Wenxuan Wendy Shi, and Brian P. Bailey. 2021. Am I Ready to Get Feedback? A Taxonomy of Factors Creators Consider Before Seeking Feedback on In-Progress Creative Work. In *Proceedings of the 13th Conference on Creativity and Cognition* (Virtual Event, Italy) *(C&C '21)*. Association for Computing Machinery, New York, NY, USA, Article 25, 10 pages. doi:10.1145/3450741.3465255

[43] Fritz Lekschas, Spyridon Ampanavos, Pao Siangliulue, Hanspeter Pfister, and Krzysztof Z. Gajos. 2021. Ask Me or Tell Me? Enhancing the Effectiveness of Crowdsourced Design Feedback. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 564, 12 pages. doi:10.1145/3411764.3445507

[44] Jie Li, Youyang Hou, Laura Lin, Ruihao Zhu, Hancheng Cao, and Abdallah El Ali. 2026. Vibe Coding in Product Teams: Reconfiguring AI-Assisted Workflows, Prototyping, and Collaboration. In *Proceedings of the 5th Annual Symposium on Human-Computer Interaction for Work* (Linz, Austria) *(CHIWORK '26)*. Association for Computing Machinery, New York, NY, USA, Article 1, 16 pages. doi:10.1145/3808045.3808062

[45] Lingyuan Li, Guo Freeman, and Nathan J. McNeese. 2022. Channeling End-User Creativity: Leveraging Live Streaming for Distributed Collaboration in Indie Game Development. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2, Article 282 (2022), 28 pages. doi:10.1145/3555173

[46] Tianyi Li, Tanay Maheshwari, and Alex Voelker. 2025. User-Centered Design with AI in the Loop: A Case Study of Rapid User Interface Prototyping with "Vibe Coding". arXiv:2507.21012 [cs.HC] https://arxiv.org/abs/2507.21012

[47] Ze Shi Li, Nowshin Nawar Arony, Kezia Devathasan, Manish Sihag, Neil A. Ernst, and Daniela E. Damian. 2024. Unveiling the Life Cycle of User Feedback: Best Practices from Software Practitioners. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 54, 13 pages. doi:10.1145/3597503.3623309

[48] Lovable Labs. 2025. Lovable: AI App Builder. Web platform. https://lovable.dev/ Accessed: 2026-09-10.

[49] Kurt Luther, Nicholas Diakopoulos, and Amy S. Bruckman. 2010. Edits & Credits: Exploring Integration and Attribution in Online Creative Collaboration. In *CHI '10 Extended Abstracts on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 2823–2832. doi:10.1145/1753846.1753869

[50] Kurt Luther, Jari-Lee Tolentino, Wei Wu, Amy Pavel, Brian P. Bailey, Maneesh Agrawala, Bjorn Hartmann, and Steven P. Dow. 2015. Structuring, Aggregating, and Evaluating Crowdsourced Design Critique. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work and Social Computing*. Association for Computing Machinery, New York, NY, USA, 473–485. doi:10.1145/2675133.2675283

[51] Thomas W. Malone, Robert Laubacher, and Chrysanthos Dellarocas. 2009. *Harnessing Crowds: Mapping the Genome of Collective Intelligence*. Technical Report 2009-001. MIT Center for Collective Intelligence, Cambridge, MA, USA. MIT Sloan Research Paper No. 4732-09.

[52] Christian Meske, Tobias Hermanns, Esther von der Weiden, Kai-Uwe Loser, and Thorsten Berger. 2025. Vibe Coding as a Reconfiguration of Intent Mediation in Software Development: Definition, Implications, and Research Agenda. *IEEE Access* 13 (2025), 213242–213259. doi:10.1109/ACCESS.2025.3645466

[53] Robert C. Miller. 2006. End-User Programming. *Commun. ACM* 49, 6 (2006), 72–76. doi:10.1145/1132469.1132487

[54] Andres Monroy-Hernandez, Benjamin Mako Hill, Jazmin Gonzalez-Rivero, and danah boyd. 2011. Computers Can't Give Credit: How Automatic Attribution Falls Short in an Online Remixing Community. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 3421–3430. doi:10.1145/1978942.1979452

[55] Lev Muchnik, Sinan Aral, and Sean J. Taylor. 2013. Social Influence Bias: A Randomized Experiment. *Science* 341, 6146 (2013), 647–651. doi:10.1126/science.1240466

[56] Thi Thao Duyen T. Nguyen, Thomas Garncarz, Felicia Ng, Laura A. Dabbish, and Steven P. Dow. 2017. Fruitful Feedback: Positive Affective Language and Source Anonymity Improve Critique Reception and Work Outcomes. In *Proceedings of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing*. Association for Computing Machinery, New York, NY, USA, 1024–1034. doi:10.1145/2998181.2998319

[57] Kevin O'Leary, Rob Gleasure, Philip O'Reilly, and Joseph Feller. 2022. Introducing the Concept of Creative Ancestry as a Means of Increasing Perceived Fairness and Satisfaction in Online Collaboration: An Experimental Study. *Technovation* 110, Article 102369 (2022), 15 pages. doi:10.1016/j.technovation.2021.102369

[58] Jonas Oppenlaender, Elina Kuosmanen, Andres Lucero, and Simo Hosio. 2021. Hardhats and Bungaloos: Comparing Crowdsourced Design Feedback with Peer Design Feedback in the Classroom. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 570, 14 pages. doi:10.1145/3411764.3445380

[59] John F. Pane, Chotirat Ann Ratanamahatana, and Brad A. Myers. 2001. Studying the Language and Structure in Non-Programmers' Solutions to Programming Problems. *International Journal of Human-Computer Studies* 54, 2 (2001), 237–264. doi:10.1006/ijhc.2000.0410

[60] Veronica Pimenova, Sarah Fakhoury, Christian Bird, Margaret-Anne D. Storey, and Madeline Endres. 2025. Good Vibrations? A Qualitative Study of Co-Creation, Communication, Flow, and Trust in Vibe Coding. arXiv:2509.12491 [cs.SE] https://arxiv.org/abs/2509.12491

[61] Adrian Preussner, Anna Crescenza, Johannes Schoning, and Florian Mathis. 2025. UrbAI: Exploring the Possibilities of Generative AI Image Processing to Promote Citizen Participation. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 21 pages. doi:10.1145/3706598.3713803

[62] Replit. 2025. Replit: AI App Builder and Publishing Platform. Web platform and documentation. https://replit.com/ Accessed: 2026-09-10.

[63] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. Association for Computing Machinery, New York, NY, USA, 491–514. doi:10.1145/3581641.3584037

[64] Elizabeth B.-N. Sanders and Pieter Jan Stappers. 2008. Co-Creation and the New Landscapes of Design. *CoDesign* 4, 1 (2008), 5–18. doi:10.1080/15710880701875068

[65] Advait Sarkar and Ian Drosos. 2025. Vibe Coding: Programming through Conversation with Artificial Intelligence. In *Proceedings of the 36th Annual Conference of the Psychology of Programming Interest Group (PPIG 2025)* (Belgrade, Serbia) *(PPIG 2025)*. Psychology of Programming Interest Group, Belgrade, Serbia, 87–118. https://ppig.org/papers/2025-ppig-36th-sarkar/

[66] Donald A. Schön. 1983. *The Reflective Practitioner: How Professionals Think in Action*. Basic Books, New York, NY, USA.

[67] Donghoon Shin, Daniel Lee, Gary Hsieh, and Gromit Yeuk-Yin Chan. 2025. PosterMate: Audience-Driven Collaborative Persona Agents for Poster Design. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology* (Busan, Republic of Korea) *(UIST '25)*. Association for Computing Machinery, New York, NY, USA, Article 201, 20 pages. doi:10.1145/3746059.3747769

[68] Ben Shneiderman. 2022. *Human-Centered AI*. Oxford University Press, Oxford, UK. doi:10.1093/oso/9780192845290.001.0001

[69] Pao Siangliulue, Joel Chan, Steven P. Dow, and Krzysztof Z. Gajos. 2016. IdeaHound: Improving Large-Scale Collaborative Ideation with Crowd-Powered Real-Time Semantic Modeling. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology* (Tokyo, Japan) *(UIST '16)*. Association for Computing Machinery, New York, NY, USA, 609–624. doi:10.1145/2984511.2984578

[70] Clay Spinuzzi. 2005. The Methodology of Participatory Design. *Technical Communication* 52, 2 (2005), 163–174.

[71] Sarah Sterman, Molly Jane Nicholas, Janaki Vivrekar, Jessica R. Mindel, and Eric Paulos. 2023. Kaleidoscope: A Reflective Documentation Tool for a User Interface Design Course. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 702, 19 pages. doi:10.1145/3544548.3581255

[72] Hari Subramonyam, Divy Thakkar, Andrew Ku, Juergen Dieber, and Anoop K. Sinha. 2025. Prototyping with Prompts: Emerging Approaches and Challenges in Generative AI Design for Collaborative Software Teams. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 882, 22 pages. doi:10.1145/3706598.3713166

[73] Xue Tan, Fujie Jin, and Alan R. Dennis. 2022. How Appreciation and Attention Affect Contributions to Electronic Networks of Practice. *Journal of Management Information Systems* 39, 4 (2022), 1037–1063. doi:10.1080/07421222.2022.2127443

[74] Thinklet. 2025. Thinklet: Social No-Code Text-to-App Platform. Web platform. https://thinklet.io/ Accessed: 2026-09-10.

[75] Jakob Tholander and Martin Jonsson. 2026. Vibe Coding Entanglements: Repositioning Boundaries of Intention, Authorship, and Responsibility in Programming with Generative AI. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems* (Barcelona, Spain) *(CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 90, 16 pages. doi:10.1145/3772318.3791847

[76] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) *(CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, Article 332, 7 pages. doi:10.1145/3491101.3519665

[77] Vercel. 2025. v0: AI App Builder. Web platform. https://v0.app/ Accessed: 2026-09-10.

[78] Fernanda B. Viégas, Martin Wattenberg, and Kushal Dave. 2004. Studying Cooperation and Conflict between Authors with History Flow Visualizations. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 575–582. doi:10.1145/985692.985765

[79] Dakuo Wang, Judith S. Olson, Jingwen Zhang, Trung Nguyen, and Gary M. Olson. 2015. DocuViz: Visualizing Collaborative Writing. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) *(CHI '15)*. Association for Computing Machinery, New York, NY, USA, 1865–1874. doi:10.1145/2702123.2702517

[80] Helen Wauck, Yu-Chun Grace Yen, Wai-Tat Fu, Elizabeth M. Gerber, Steven P. Dow, and Brian P. Bailey. 2017. From in the Class or in the Wild? Peers Provide Better Design Feedback Than External Crowds. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 5580–5591. doi:10.1145/3025453.3025477

[81] Yong Wu and Christian D. Schunn. 2020. From Feedback to Revisions: Effects of Feedback Features and Perceptions. *Contemporary Educational Psychology* 60 (2020), 101826. doi:10.1016/j.cedpsych.2019.101826

[82] Anbang Xu and Brian P. Bailey. 2012. What Do You Think? A Case Study of Benefit, Expectation, and Interaction in a Large Online Critique Community. In *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work (CSCW '12)*. Association for Computing Machinery, New York, NY, USA, 295–304. doi:10.1145/2145204.2145252

[83] Anbang Xu, Shih-Wen Huang, and Brian P. Bailey. 2014. Voyant: Generating Structured Feedback on Visual Designs Using a Crowd of Non-Experts. In *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work and Social Computing* (Baltimore, Maryland, USA) *(CSCW '14)*. Association for Computing Machinery, New York, NY, USA, 1433–1444. doi:10.1145/2531602.2531604

[84] Anbang Xu, Huaming Rao, Steven P. Dow, and Brian P. Bailey. 2015. A Classroom Study of Using Crowd Feedback in the Iterative Design Process. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work and Social Computing* (Vancouver, BC, Canada) *(CSCW '15)*. Association for Computing Machinery, New York, NY, USA, 1637–1648. doi:10.1145/2675133.2675140

[85] Lixiang Yan, Vanessa Echeverria, Gloria Milena Fernandez Nieto, Yueqiao Jin, Zachari Swiecki, Linxuan Zhao, Dragan Gasevic, and Roberto Martinez-Maldonado. 2024. Human-AI Collaboration in Thematic Analysis Using ChatGPT: A User Study and Design Recommendations. In *Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 191, 7 pages. doi:10.1145/3613905.3650732

[86] Yu-Chun Grace Yen, Steven P. Dow, Elizabeth M. Gerber, and Brian P. Bailey. 2017. Listen to Others, Listen to Yourself: Combining Feedback Review and Reflection to Improve Iterative Design. In *Proceedings of the 2017 ACM SIGCHI Conference on Creativity and Cognition* (Singapore, Singapore) *(C&C '17)*. Association for Computing Machinery, New York, NY, USA, 158–170. doi:10.1145/3059454.3059468

[87] Meredith Young-Ng, Qingxiaoyang Zhu, Jingxian Liao, and Hao-Chuan Wang. 2025. Balancing Human Agency and AI Autonomy in Human-AI Idea Selection. In *Extended Abstracts of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 91, 6 pages. doi:10.1145/3706599.3719880

[88] Chao Zhang, Kexin Ju, Peter Bidoshi, Yu-Chun Grace Yen, and Jeffrey M. Rzeszotarski. 2025. Friction: Deciphering Writing Feedback into Writing Revisions through LLM-Assisted Reflection. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 935, 27 pages. doi:10.1145/3706598.3714316

[89] Chao Zhang, Kexin Ju, Zhuolun Han, Yu-Chun Grace Yen, and Jeffrey M. Rzeszotarski. 2025. Synthia: Visually Interpreting and Synthesizing Feedback for Writing Revision. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, New York, NY, USA, Article 88, 16 pages. doi:10.1145/3746059.3747703

[90] Chu Zhang, XiaoKe Zeng, Jin Zhang, Ruoyu Wen, Vince Siu, Richard William Allen, and Ray LC. 2026. FIERO: Empowering Creative Writing through Collaborative Game Play. arXiv:2607.11837 [cs.HC] https://arxiv.org/abs/2607.11837

[91] Shuning Zhang, Hui Wang, and Xin Yi. 2025. Exploring Collaboration Patterns and Strategies in Human-AI Co-creation through the Lens of Agency: A Scoping Review of the Top-tier HCI Literature. *Proceedings of the ACM on Human-Computer Interaction* 9, 7, Article CSCW413 (2025), 43 pages.

doi:10.1145/3757594